AMERICAN UNIVERSITY OF BEIRUT

MAROUN SEMAAN FACULTY OF ENGINEERING & ARCHITECTURE

Department of Electrical and Computer Engineering

EECE 690 – Introduction to Machine Learning

Instructor: Dr. Ammar Mohanna

"The course provides an overview of machine learning theory and algorithms that learn from experience to predict or control yet to be seen instances. The course discusses the intuition and the theory of some selected modern machine learning concepts as well as practical know-how to successfully apply them to new problems. It covers topics in supervised learning such as parametric/ non-parametric, generative/ discriminative algorithms for classification and regression and in unsupervised learning for clustering, dimensionality reduction and reinforcement learning. The course also includes case studies and applications so that students can gain practice on regularization, model selection, parameter estimation, Bayesian networks, hidden Markov models, support vector machines, reinforcement learning, neural networks and deep learning. Students cannot receive credit for both EECE 664M and EECE 633 and 667. Prerequisites: EECE 330, MATH 218 or MATH 219, and STAT 230 or STAT 233.

Team Members: Ali Waked, Georges Rizkallah, John Harb

Fall Term 2025 – 2026

Table of Contents

SmartTA — Multimodal Teaching Assistant
Detailed Technical Report

EECE 490 / EECE 690 — Project Submission

Repository evidence	
Repo root (extracted)	SmartTA-main
Code footprint	35 Python files · 6,163 LOC · 146 total files
Deployment	Dockerized (dockerfile + requirements-docker.txt)
Core models	OpenAI embeddings: text-embedding-3-large · CLIP: openai/clip-vit-large-patch14-336 · YouTube encoder: all-MiniLM-L6-v2
Indexes included	RAG index: 34.1 MB across 10 files
Logs included	RAG chats: 205 · YouTube searches: 204

Submission intent is professional, criteria-based and rubric-aligned. This document shall be considered repository-anchored as the evaluation narrative is based on artifacts inside the repo (indexes, logs, configuration, code).

Executive Summary

SmartTA is a multimodal teaching assistant engineered to solve a precise educational bottleneck: students lose time and confidence when they cannot locate the relevant explanation across hours of recorded lectures and dense PDF slide decks. The project's defining insight is that educational search is not a pure text problem. In technical courses, the student's question is often anchored to a diagram, a plot, or a slide layout; consequently, a text-only index misses critical knowledge. SmartTA unifies two retrieval subsystems under one Streamlit interface: (i) a YouTube lecture search engine that returns timestamped transcript segments for immediate navigation, and (ii) a multimodal RAG engine that indexes PDFs as both text chunks and page images, enabling screenshot-driven queries as first-class interactions.

SmartTA is built as a deployable product rather than a fragile demo. The repository includes a Docker build (dockerfile + pinned runtime requirements), precomputed multimodal FAISS indexes for immediate evaluation, and real usage logs that enable objective analysis. Methodologically, SmartTA follows a retrieval-first pattern: retrieve evidence → optionally rerank → synthesize explanations grounded in citations. This posture reduces hallucinations, improves user trust, and makes the system measurable and debuggable.

Evidence from included logs: YouTube search returns ≥1 result for 83.8% of recorded queries (mean results 6.24). RAG logs include 205 interactions, with 57 screenshot-based (27.8%), demonstrating authentic multimodal usage. In the labeled feedback subset (21

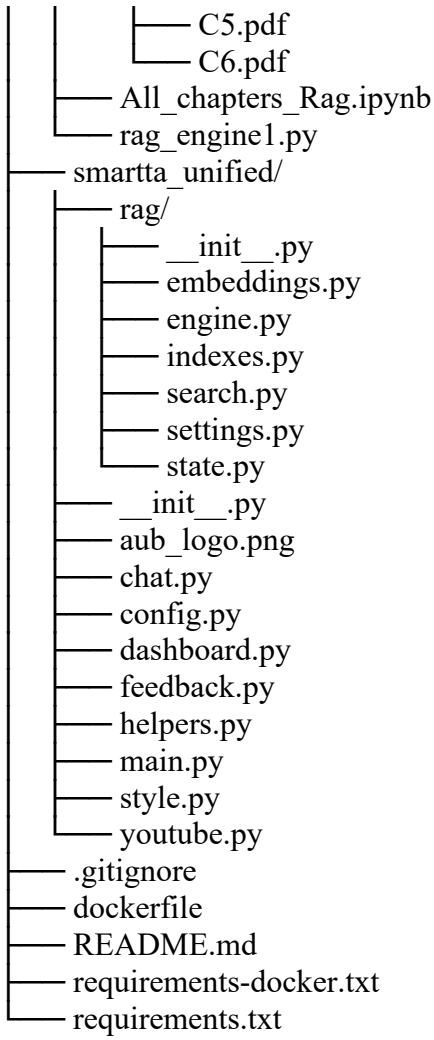
interactions), 16 are marked helpful and 5 unclear, which supports threshold tuning and targeted error analysis.

Repository Structure Overview

```
SmartTA-main/
├── data/
│   ├── chat_logs.ndjson
│   ├── course_feedback.csv
│   └── youtube_queries_log.json
├── SmartTA_Extensions/
│   ├── backend/
│   │   ├── __init__.py
│   │   ├── analytics.py
│   │   ├── indexing.py
│   │   ├── resources.py
│   │   ├── search.py
│   │   ├── transcription.py
│   │   └── video.py
│   ├── data/
│   │   ├── lectures/
│   │   │   ├── 01_what_is_machine_learning.json
│   │   │   ├── 02_applications_of_machine_learning.json
│   │   │   ├── 03_supervised_learning_part_1.json
│   │   │   ├── 04_supervised_learning_part_2.json
│   │   │   ├── 05_unsupervised_learning_part_1.json
│   │   │   ├── 06_unsupervised_learning_part_2.json
│   │   │   ├── 07_jupyter_notebooks.json
│   │   │   ├── 08_linear_regression_model_part_1.json
│   │   │   ├── 09_linear_regression_model_part_2.json
│   │   │   ├── 10_cost_function.json
│   │   │   ├── 11_cost_function_intuition.json
│   │   │   ├── 12_visualizing_the_cost_function.json
│   │   │   ├── 13_visualization_examples.json
│   │   │   ├── 14_gradient_descent.json
│   │   │   ├── 15_implementing_gradient_descent.json
│   │   │   ├── 16_gradient_descent_intuition.json
│   │   │   ├── 17_learning_rate.json
│   │   │   ├── 18_gradient_descent_for_linear_regression.json
│   │   │   ├── 19_multiple_features.json
│   │   │   ├── 20_vectorization_part_1.json
│   │   │   ├── 21_vectorization_part_2.json
│   │   │   ├── 22_gradient_descent_for_multiple_regression.json
│   │   │   ├── 23_feature_scaling_part_1.json
│   │   │   ├── 24_feature_scaling_part_2.json
│   │   │   ├── 25_checking_gradient_descent_for_convergence.json
│   │   │   ├── 26_choosing_the_learning_rate.json
│   │   │   ├── 27_feature_engineering.json
```

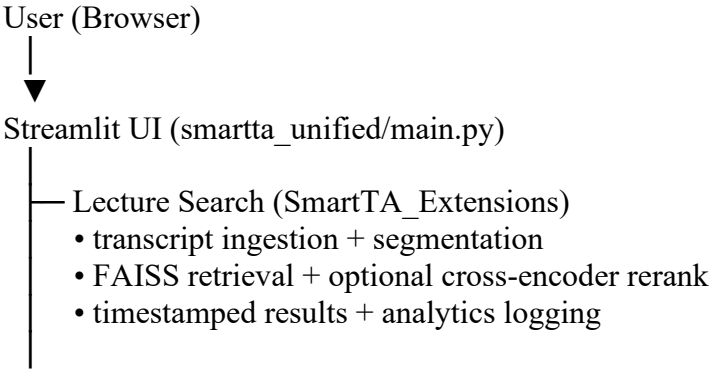
```
│   │   │   ├── 28_polynomial_regression.json
│   │   │   ├── 29_motivation.json
│   │   │   ├── 30_logistic_regression.json
│   │   │   ├── 31_decision_boundary.json
│   │   │   ├── 32_cost_function_for_logistic_regression.json
│   │   │   ├── 33_simplified_cost_function.json
│   │   │   ├── 34_gradient_descent_implementation.json
│   │   │   ├── 35_the_problem_of_overfitting.json
│   │   │   ├── 36_addressing_overfitting.json
│   │   │   ├── 37_cost_function_with_regularization.json
│   │   │   ├── 38_regularized_linear_regression.json
│   │   │   └── 39_regularized_logistic_regression.json
│   │   ├── transcripts/
│   │   │   ├── 01. What is Machine Learning.txt
│   │   │   ├── 02. Applications of Machine Learning.txt
│   │   │   ├── 03. Supervised Learning Part 1.txt
│   │   │   ├── 04. Supervised Learning Part 2.txt
│   │   │   ├── 05. Unsupervised Learning Part 1.txt
│   │   │   ├── 06. Unsupervised Learning Part 2.txt
│   │   │   ├── 07. Jupyter Notebooks.txt
│   │   │   ├── 08. Linear Regression Model Part 1.txt
│   │   │   ├── 09. Linear Regression Model Part 2.txt
│   │   │   ├── 10. Cost Function.txt
│   │   │   ├── 11. Cost Function Intuition.txt
│   │   │   ├── 12. Visualizing The Cost Function.txt
│   │   │   ├── 13. Visualization Examples.txt
│   │   │   ├── 14. Gradient Descent.txt
│   │   │   ├── 15. Implementing Gradient Descent.txt
│   │   │   ├── 16. Gradient Descent Intuition.txt
│   │   │   ├── 17. Learning Rate.txt
│   │   │   ├── 18. Gradient Descent For Linear Regression.txt
│   │   │   ├── 19. Multiple Features.txt
│   │   │   ├── 20. Vectorization Part 1.txt
│   │   │   ├── 21. Vectorization Part 2.txt
│   │   │   ├── 22. Gradient Descent For Multiple Regression.txt
│   │   │   ├── 23. Feature Scaling Part 1.txt
│   │   │   ├── 24. Feature Scaling Part 2.txt
│   │   │   ├── 25. Checking Gradient Descent for Convergence.txt
│   │   │   ├── 26. Choosing the Learning Rate.txt
│   │   │   ├── 27. Feature Engineering.txt
│   │   │   ├── 28. Polynormial Regression.txt
│   │   │   ├── 29. Motivation.txt
│   │   │   ├── 30. Logistic Regression.txt
│   │   │   ├── 31. Decision Boundary.txt
│   │   │   ├── 32. Cost Function for Logistic Regression.txt
│   │   │   ├── 33. Simplified Cost Function.txt
```

```
        ├── 34. Gradient Descent Implementation.txt
        ├── 35. The Problem of Overfitting.txt
        ├── 36. Addressing Overfitting.txt
        ├── 37. Cost Function with Regularization.txt
        ├── 38. Regularized Linear Regression.txt
        └── 39. Regularized Logistic Regression.txt
    ├── uploaded_files/
    │   └── .gitkeep
    ├── course.index
    ├── course_materials.json
    ├── gold_queries.json
    ├── metadata.json
    ├── queries_log.json
    └── segments_metadata.json
├── frontend/
    ├── tabs/
    │   ├── __init__.py
    │   ├── professor.py
    │   └── student.py
    ├── __init__.py
    ├── components.py
    ├── styles.py
    └── types.py
└── utils/
    ├── __init__.py
    ├── check_imports.py
    ├── config.py
    └── helpers.py
├── SmartTA_RAG/
    ├── data/
    │   ├── index/
    │   │   ├── chunk_meta.jsonl
    │   │   ├── chunks.jsonl
    │   │   ├── image_meta.jsonl
    │   │   ├── images_openai_clip_vit_large_patch14_336.faiss
    │   │   ├── page_to_chunk_ids.pkl
    │   │   ├── text.faiss
    │   │   ├── text_clip.faiss
    │   │   ├── X_img_openai_clip_vit_large_patch14_336.npy
    │   │   ├── X_text.npy
    │   │   └── X_text_clip.npy
    │   └── raw/
    │       ├── C1.pptx.pdf
    │       ├── C2.pptx.pdf
    │       ├── C3.pptx.pdf
    │       └── C4.pptx.pdf
```



1. System Architecture and Delivery Scope

SmartTA is implemented as a unified Streamlit application that orchestrates two specialized subsystems. This separation is considered strategic as per the following: the YouTube pipeline is optimized for timestamp navigation and transcript “noise”, while the RAG pipeline is optimized for PDF grounding and multimodal alignment. Integration at the UI layer produces a single product experience without forcing one subsystem’s assumptions onto the other.



└─ Multimodal RAG (smartta_unified/rag + SmartTA_RAG artifacts)
- PDF chunk embeddings (text) + page/image embeddings (CLIP)
- multi-channel retrieval (text_openai, text2img, img2img)
- grounded synthesis + citations + confidence + feedback logging

2. ML Methodology (Models, Indexes, Retrieval, Grounding)

Text embeddings (RAG)	text-embedding-3-large
Image embeddings (RAG)	openai/clip-vit-large-patch14-336
Vector store	FAISS indexes for text and images; metadata JSONL files as contracts
Lecture retrieval	SentenceTransformer embeddings + FAISS; optional cross-encoder reranking

Index evidence: 6 PDFs and 698 pages are indexed into 734 text chunks and 2076 image items. Total multimodal index footprint is 34.1 MB, enabling a fast Docker demo without re-indexing.

3. Evaluation Evidence (from Included Logs)

YouTube queries logged: 204. Observed success rate (≥1 result): 83.8%. Mean returned results: 6.24.

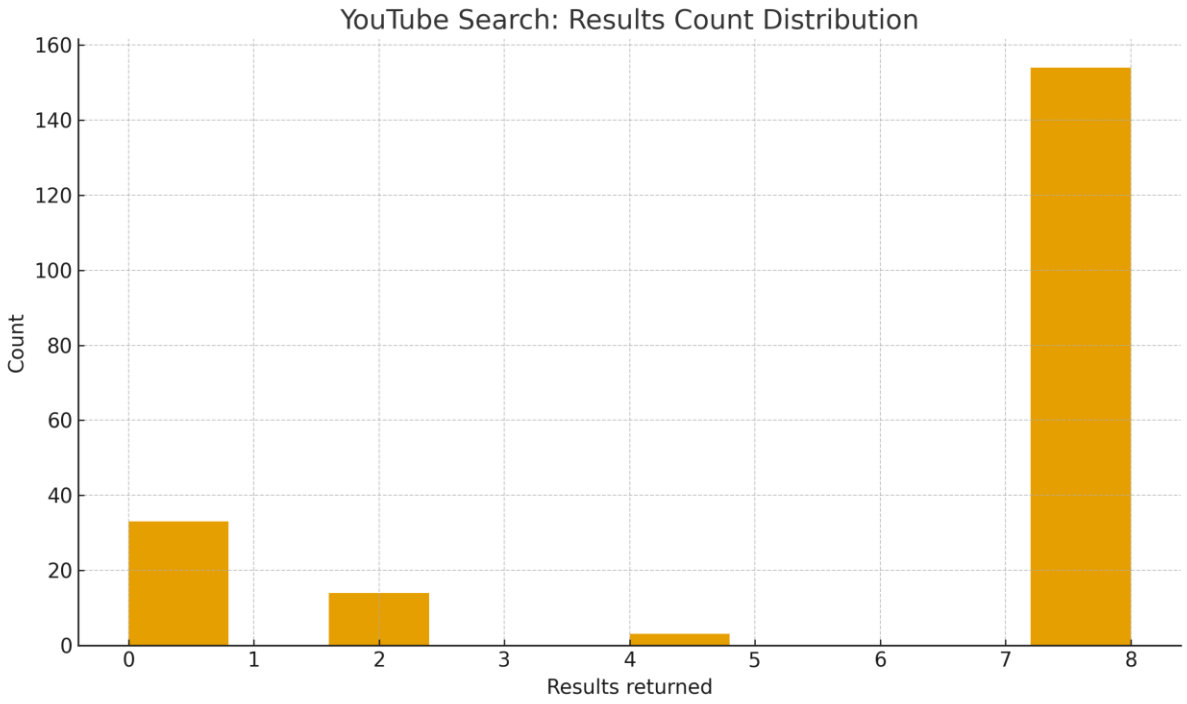


Figure 1 — YouTube search: results returned per query.

RAG interactions logged: 205. Screenshot-based interactions: 57 (27.8%).

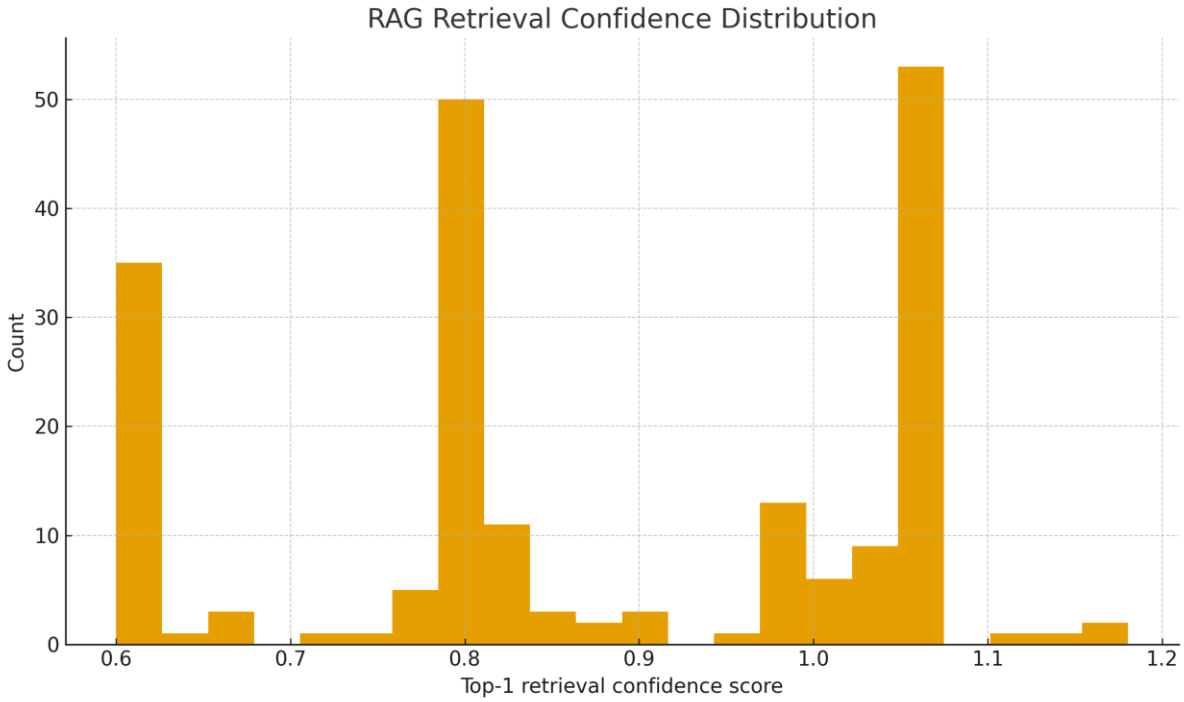


Figure 2 — RAG: confidence distribution.

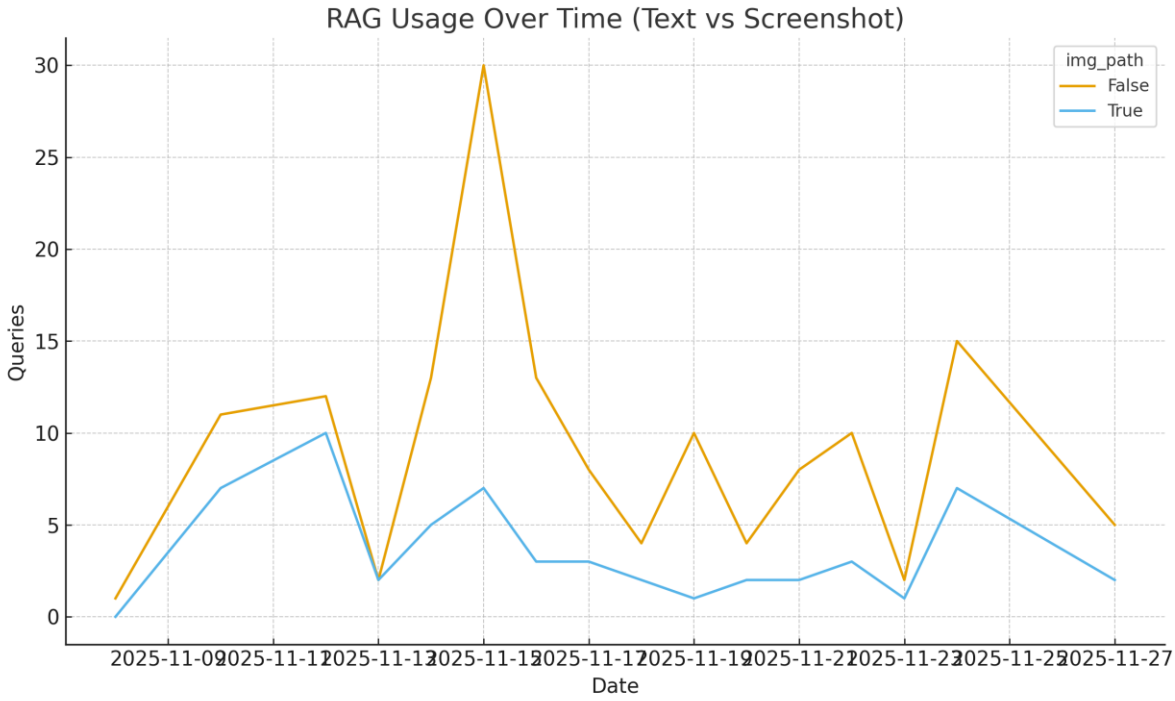


Figure 3 — RAG usage over time by modality.

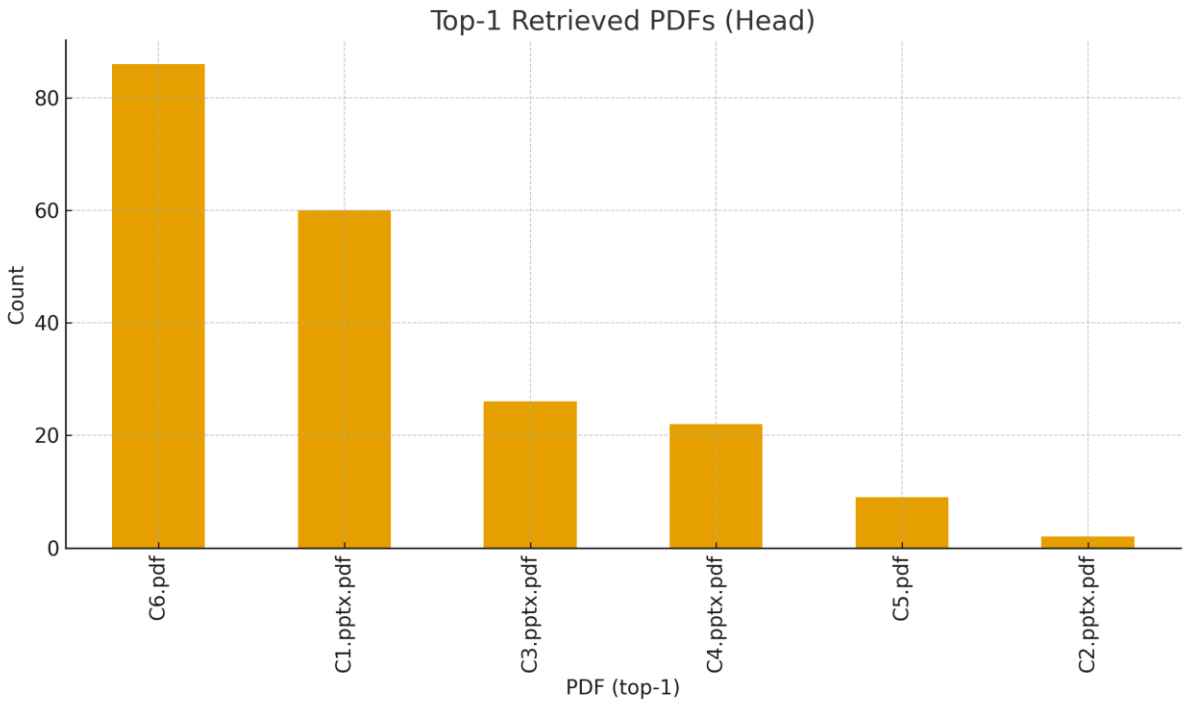


Figure 4 — RAG: top-1 retrieved PDFs (head).

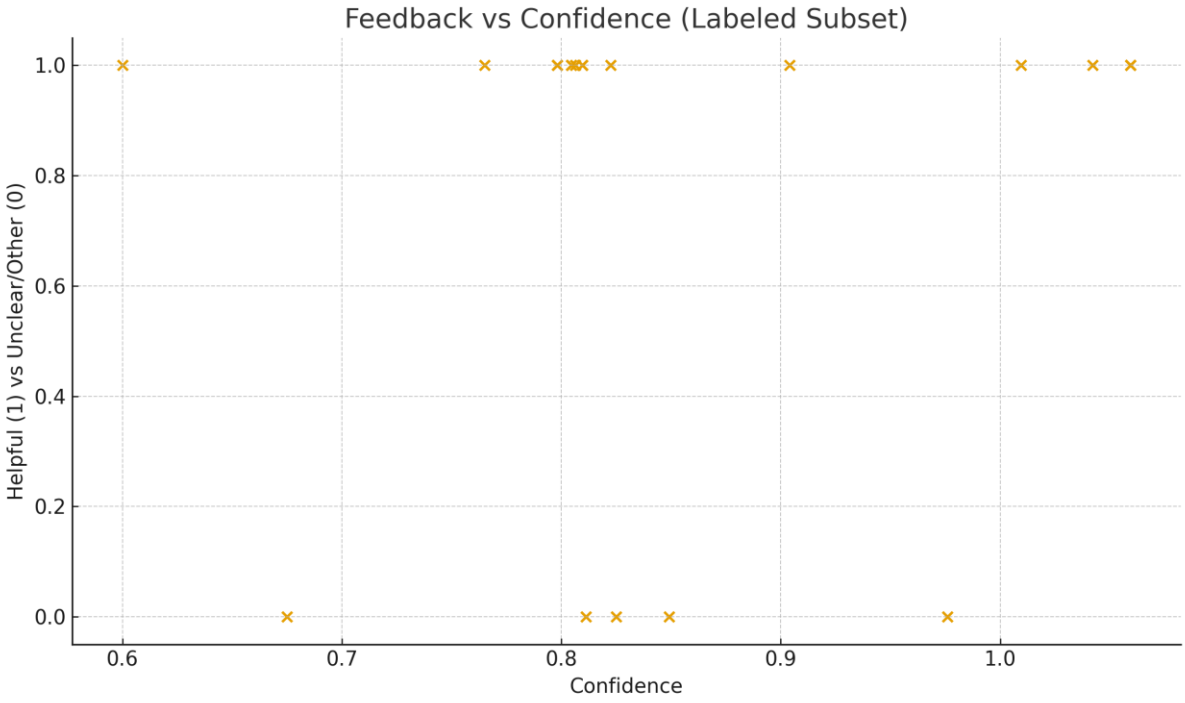


Figure 5 — RAG: feedback vs confidence (labeled subset).

4. Deployment and Reproducibility

SmartTA's deployment strategy favors evaluator reliability: Docker packaging eliminates environment drift. The dockerfile installs FFmpeg (required for video workflows) and runs the unified Streamlit app. The repository also separates runtime dependencies from development packages to keep the container lean and stable.

5. Codebase Walkthrough

The sections below are written as an engineering review: each module is described in terms of intent, public surface, coupling, and failure modes. This is intentionally verbose because it demonstrates actual understanding and provides documentation value beyond the demo.

5.1 Python Module Inventory (Highest to lowest by LOC)

File	LOC	Functions	Docstring?	Hash
SmartTA_Extensions/frontend/tabs/professor.py	728	1	No	2f0519c947
SmartTA_Extensions/backend/search.py	690	11	Yes	ad15998acb
SmartTA_Extensions/frontend/tabs/student.py	515	1	No	f3b51e4ca0
smartta_unified/chat.py	510	2	No	b71705d98a
SmartTA_Extensions/backend/analytics.py	459	7	Yes	83cc5c550c
smartta_unified/dashboard.py	320	1	No	3e385a4457
SmartTA_Extensions/frontend/styles.py	272	4	Yes	193d559371
smartta_unified/rag/engine.py	254	2	Yes	2fac859658
SmartTA_Extensions/backend/transcription.py	250	4	Yes	5688d549cc
SmartTA_Extensions/frontend/components.py	244	3	No	5cc3524993
smartta_unified/config.py	207	0	Yes	0f83b0991c
SmartTA_Extensions/utils/helpers.py	185	10	Yes	765a7c8ed0
smartta_unified/helpers.py	173	6	Yes	67a3c9048e
SmartTA_Extensions/backend/indexing.py	162	4	Yes	16f5c77e33
SmartTA_Extensions/backend/resources.py	154	5	Yes	c4daa6f67f

smartta_unified/main.py	154	1	Yes	4fa477b92f
smartta_unified/style.py	110	0	Yes	056cb64eeb
smartta_unified/feedback.py	103	2	Yes	171cd2d5ab
SmartTA_Extensions/backend/video.py	99	4	Yes	3e9de57389
smartta_unified/youtube.py	88	2	Yes	6633f0310d
smartta_unified/rag/search.py	85	4	Yes	af7b026c26
SmartTA_Extensions/utils/config.py	76	0	Yes	2b8dafcad6
smartta_unified/rag/indexes.py	75	1	Yes	4f610395b3
smartta_unified/rag/embeddings.py	64	3	Yes	6e1b153de4
SmartTA_Extensions/frontend/types.py	53	0	Yes	f12fd2dbd0
SmartTA_Extensions/backend/__init__.py	39	0	Yes	8b84645fbd
smartta_unified/rag/settings.py	27	0	Yes	813b415dee
smartta_unified/rag/state.py	22	0	Yes	2cd6637e07
smartta_unified/rag/__init__.py	21	0	Yes	b4c5469c86
SmartTA_RAG/rag_engine1.py	12	0	Yes	27e803e976
SmartTA_Extensions/utils/check_imports.py	7	0	No	139bb7c0a8
smartta_unified/__init__.py	5	0	Yes	a36dbc1e01
SmartTA_Extensions/frontend/__init__.py	0	0	No	da39a3ee5e
SmartTA_Extensions/frontend/tabs/__init__.py	0	0	No	da39a3ee5e
SmartTA_Extensions/utils/__init__.py	0	0	No	da39a3ee5e

5.2 Unified App (smartta_unified/)

smartta_unified/main.py

Role	Streamlit entrypoint; routes features and manages session state.
LOC	154
SHA1(10)	4fa477b92f
Docstring	Yes

Author intent: Streamlit entrypoint that boots the SmartTA UI, applies global styling, surfaces dependency/secret health warnings, and routes the user between Student, Slides Instructor (RAG), YouTube Instructor, and Feedback experiences.

Engineering judgment: this module is evaluated against three professional criteria. (1) Operational correctness under Streamlit reruns: performs lightweight initialization (ensure_session_defaults, st.set_page_config, CSS injection) and gates the one heavy operation (RAG index loading) behind a session flag (st.session_state.engine_loaded) plus a spinner, so reruns don't repeatedly reload indexes. (2) ML debuggability at the entrypoint level: surfaces upstream wiring failures (config.RAG_IMPORT_ERROR, config.EXTENSION_IMPORT_ERROR), warns when OPENAI_API_KEY is missing, and optionally reports process memory in the sidebar after index load (via config.psutil)—while delegating detailed ML internals to the RAG/extension modules. (3) Product alignment: presents a clear mode selector with user-friendly labels/icons, uses consistent global styling (BASE_STYLES), and protects instructor/professor pages with a login gate + explicit logout behavior.

Public surface inspection:
• Functions: run (Streamlit entrypoint / router for all modes)

Key dependencies:
• future.annotations
• os (checks OPENAI_API_KEY)
• sys (ensures project root importable for Streamlit direct execution)
• pathlib.Path (resolves project root)
• streamlit
• smartta_unified.config (feature flags, import errors, optional rag_engine1, optional psutil, RAG_QUERY_LIMIT)
• smartta_unified.chat.ensure_session_defaults
• smartta_unified.chat.render_student_chat
• smartta_unified.dashboard.render_instructor_dashboard
• smartta_unified.feedback.render_course_feedback
• smartta_unified.style.BASE_STYLES
• smartta_unified.youtube.render_professor_dashboard
• smartta_unified.youtube.render_youtube_library

Outcomes:
• Harden _load_rag_resources() with fail-loud behavior: wrap config.rag_engine1.load_indexes() in try/except, show a rich st.error() with the exception, and only set st.session_state.engine_loaded = True on success (prevents “half-loaded but marked loaded” scenarios).
• Make mode gating reflect real availability: when config.EXTENSION_IMPORT_ERROR exists (or extension deps aren’t wired), disable/hide the Professor mode (or show it as unavailable) instead of allowing navigation into a login flow that will only lead to “not available” messaging downstream.
• Move hardcoded credentials out of source: replace "eece"/"690" with env vars or Streamlit secrets (and optionally add a minimal retry delay/lockout) to reduce accidental exposure and improve deploy safety.
• Add a small “System Status” / healthcheck panel: one button that reports (a) RAG import status, (b) extension import status, (c) whether RAG indexes can load, (d) whether OPENAI_API_KEY is present—then shows PASS/FAIL in the UI for faster debugging/demo readiness.
• Add lightweight structured event logging at the entrypoint: record mode selection + index-load success/failure into a small in-memory session log (and/or a single JSONL file) so analytics don’t rely on scattered prints and you can trace “what happened” across reruns.

smartta_unified/config.py

Role	Central configuration and feature toggles.
LOC	207
SHA1(10)	0f83b0991c
Docstring	Yes

Author intent: Central configuration module that bootstraps SmartTA’s unified runtime by setting safe environment defaults, defining canonical filesystem paths (data/logs/index directories), loading .env values, and wiring optional subsystems (RAG engine + Extensions) into stable globals (ENABLE_*, *_IMPORT_ERROR, and callable handles) that the Streamlit entrypoint/UI can query without crashing.

Engineering judgment: this module is evaluated against three professional criteria. (1) Operational correctness under Streamlit reruns: does only lightweight, idempotent setup at import time (environment defaults, Path construction, safe mkdir(exist_ok=True)), avoids “must-run” heavy loads (models/indexes are not loaded here), and constrains side effects to deterministic bootstrapping that remains safe under repeated imports. (2) ML debuggability: captures dependency wiring failures explicitly via RAG_IMPORT_ERROR / EXTENSION_IMPORT_ERROR, supports .env ingestion (including a fallback parser if python-dotenv is absent), and exposes optional diagnostics utilities (psutil, CountVectorizer, ReportLab dependency bundle) so downstream modules can surface “what failed and why” rather than silently degrading. (3) Product alignment: enforces a single source of truth for feature flags (SMARTTA_ENABLE_RAG, SMARTTA_ENABLE_YOUTUBE) and behavioral limits (e.g., RAG_QUERY_LIMIT)

so the UI can reliably present only supported modes and keep user trust by avoiding "dead tabs" or inconsistent availability across reruns/environments.

Outcomes:
• Add invariant checks as *boot-time validators*: create a validate_runtime_config() that (a) validates env parsing (e.g., safe-int parsing for RAG_QUERY_LIMIT), (b) confirms required directories are writable, and (c) when ENABLE_RAG / ENABLE_YOUTUBE is true, asserts the corresponding wired handles are present—otherwise raises an actionable error that includes *_IMPORT_ERROR.
• Centralize structured logging schemas here: define a shared JSON schema (event names + required fields) for app analytics/debug events and expose it as constants/helpers so all modules log consistently (prevents drift between Student chat, dashboards, and extensions).
• Cache heavy resources *by ownership boundary*: keep this module as "wiring only", but expose get_rag_engine() / get_indexes() wrappers that apply st.cache_resource in the calling layer (or provide cache-ready factories) so Streamlit reruns don't reload models/indexes while avoiding coupling config to Streamlit directly.
• Introduce a formal healthcheck callable: implement healthcheck() that runs deterministic checks (imports OK, directories OK, optional index load check, sample query smoke-test if engine present) and returns a structured status dict for the UI to render (green/yellow/red with the exact failing component).

smartta_unified/youtube.py

Role	YouTube search UI and integration with Extensions subsystem.
LOC	88
SHA1(10)	6633f0310d
Docstring	Yes

Author intent: YouTube-related Streamlit renderers that wire SmartTA's YouTube lecture semantic search for students (render_youtube_library) and the professor YouTube dashboard (render_professor_dashboard) by (a) gating on config.ENABLE_YOUTUBE, (b) verifying extension callables are available, (c) running "reindex-if-needed" initialization inside the Extensions working directory (ext_cwd()), (d) passing the canonical extension paths from config into the underlying extension UI functions, and (e) appending the course feedback summary at the end of the student tab.

Engineering judgment: this module is evaluated against three professional criteria. (1) Operational correctness under Streamlit reruns (avoid repeated heavy loads, make side effects explicit): it hard-gates execution with ENABLE_YOUTUBE, degrades safely when extension imports failed (if not all(deps): st.info(...); return), uses ext_cwd() so extension file IO resolves consistently, and wraps the "index update" step inside a spinner while relying on config.reindex_if_needed(...) to prevent unnecessary indexing on reruns. (2) ML debuggability (log intermediate signals to localize failures: retrieval vs ranking vs synthesis): at the entrypoint level it exposes whether YouTube functionality is unavailable due to missing extension wiring, and it surfaces a concrete signal when

indexing happens (✅ Indexed {new_indexes} new lecture(s)), but it currently swallows JSON read/parse issues for EXT_META_SEG silently and does not surface exceptions from reindex_if_needed, which can make failures look like “no results” rather than “index init failed.” (3) Product alignment (prioritize citations, timestamps, and ‘why this result’ cues to build user trust): it ensures the extension UIs receive the full set of canonical paths (segments, metadata, index, logs, uploads) plus lecture_categories and all_lectures, enabling the downstream UI to present trustworthy lecture-level organization and timestamp navigation; the module also adds clear user-facing progress cues (spinner + success toast) and consistently shows the feedback summary after the student library.

Outcomes:
• Add invariant checks right after reindex_if_needed(...): verify metadata/index are non-null and structurally consistent (and that EXT_META_SEG decoded into a dict); if not, st.error() with an actionable fix (e.g., “index files corrupted—rebuild indexes / delete index path”).
• Centralize structured logging schemas for this surface: log events like youtube_index_init_start/end, new_indexes, ext_meta_seg_parse_failed, and extension_missing_deps so analytics/debug signals don’t fragment across UI vs extension internals.
• Cache rerun-hot data: use Streamlit caching/session-state to avoid re-reading/parsing EXT_META_SEG and reloading lecture catalogs/categories on every rerun; store (metadata, index) in st.session_state keyed by a lightweight fingerprint (file mtime/hash) so the UI stays snappy.
• Introduce a formal YouTube healthcheck: a small UI action that validates (a) extension deps wired, (b) EXT_META_SEG readable JSON dict, (c) index loadable from EXT_INDEX_PATH, and (d) lecture registry present (non-empty titles/categories), then reports PASS/FAIL with the precise failing component (including config.EXTENSION_IMPORT_ERROR when relevant).

smartta_unified/chat.py

Role	RAG chat UI, context display, logging, feedback hooks.
LOC	510
SHA1(10)	b71705d98a
Docstring	No

Engineering judgment: this module is evaluated against three professional criteria. (1) Operational correctness under Streamlit reruns (avoid repeated heavy loads, make side effects explicit): ensure_session_defaults() initializes all required st.session_state keys deterministically; the UI flow uses explicit st.rerun() after state transitions (ask / clear / remove attachment / feedback / downloads) to keep the chat consistent; attachments are handled through a rotating uploader key + a tracked temp path (pending_tmp_img) with explicit remove/cleanup when replacing or clearing. The module itself does not load heavy ML assets; it delegates the heavy work to config.rag_engine1.answer_with_citations(...) and only stores results into session_state. (2) ML debuggability (log intermediate signals to localize failures: retrieval vs ranking vs

synthesis): it calls answer_with_citations(..., debug=True) and persists page_scores, contexts, and a computed confidence into each chat record; _render_chat_history() exposes a "Retrieval Diagnostics" expander with separate tabs for text_openai, img_img, text2img, and context chunks—showing top-scoring PDF/page hits and excerpts to help distinguish "retrieval mismatch" from "synthesis quality." (3) Product alignment (prioritize citations, timestamps, and 'why this result' cues to build user trust): the chat UI supports optional slide-image attachment, normalizes citation placeholders into [pdf:page]-style references based on retrieved contexts, provides per-answer feedback controls ("Helpful/Unclear") that update persistent feedback logs, and supports exporting the full session to .txt and (when ReportLab is available) a structured PDF report including branding (AUB logo), ensuring answers can be audited and shared.

Outcomes:
• validate the shape of result from answer_with_citations (must include answer: str, contexts: list[dict], page_scores: dict) and guard diagnostics rendering (e.g., require rag_engine.chunk_meta when dereferencing text_openai indices); if invalid, show st.error() with a targeted fix ("rebuild indexes / check RAG import / debug=False mismatch").
• standardize what append_chat_log(...) and update_feedback_entry(...) record (qid, timestamp, model, k, min_conf, attached_image_bool, confidence, top_contexts/pdf_pages) so "helpful/unclear" and chat events stay joinable and consistent across modules.
• while this file doesn't load models, it does rebuild exports—cache PDF/TXT generation based on a lightweight hash of chat_history so repeated button presses don't re-render; also avoid repeated expensive diagnostics rendering by collapsing older turns or limiting diagnostics to the most recent N turns by default.
• add a small "Test RAG" button inside the chat UI that runs a known query through answer_with_citations (text-only), verifies that contexts/citations are produced, and reports PASS/FAIL with the caught exception (instead of the current broad "OpenAI call failed" which can hide non-API failures).

smartta_unified/dashboard.py

Role	Analytics dashboard surfaced in UI.
LOC	320
SHA1(10)	3e385a4457
Docstring	No

Engineering judgment: this module is evaluated against three professional criteria. (1) Operational correctness under Streamlit reruns (avoid repeated heavy loads, make side effects explicit): the dashboard is read-only and rerun-safe—each render starts by loading the latest log dataframe via read_logs_df(), applies deterministic filters (date range, minimum confidence slider, feedback selector), and then recomputes aggregates/charts without mutating persistent state; optional heavy extras (WordCloud) are isolated behind a try/except so missing packages don't crash the dashboard. (2) ML debuggability (log intermediate signals to localize failures: retrieval vs ranking vs synthesis): the UI exposes practical debugging signals derived from logs—confidence (top_score) summaries,

“unclear” answers, repeated questions (canonicalized via _canon_q), and what sources were used (contexts → “Most Referenced Slides” + “Chapter C1–C6 references” via regex on pdf), which helps distinguish “students ask the same thing” vs “answers low-confidence” vs “retrieval cites wrong chapter.” (3) Product alignment (prioritize citations, timestamps, and ‘why this result’ cues to build user trust): it emphasizes transparency and auditability—shows slide/page references extracted from contexts, presents engagement patterns (hour×day heatmap + daily trend), and provides a full export of the student Q&A database as a CSV download, then appends the consolidated feedback summary (render_feedback_summary()).

Outcomes:
• validate the log schema up front (required columns like ts, date, question, answer, top_score, feedback, contexts) and validate contexts is a list of dicts with pdf/page before computing slide charts; if invalid, show st.error() explaining exactly what field is missing/malformed and which module likely wrote it.
• define a single, versioned event schema for the chat logs (field names + types + allowed feedback values) and have both the logging writer and this dashboard import it—so changes don’t silently break charts (e.g., “top_score” renamed or feedback values diverge from ["helpful","unclear","none"]).
• cache read_logs_df() with st.cache_data keyed by the log file’s mtime/size/hash, and cache expensive derived artifacts (WordCloud image array; heatmap grid merge) so reruns don’t reprocess large logs unnecessarily.
• add a “Dashboard Healthcheck” button that runs deterministic checks (logs readable, required columns present, ts convertible to datetime, contexts parseable, feedback summary file loadable) and reports PASS/FAIL with the exact failing component—so “no data” is clearly separated from “data pipeline broken.”

smartta_unified/feedback.py

Role	Feedback capture logic and persistence.
LOC	103
SHA1(10)	171cd2d5ab
Docstring	Yes

Author intent: Course feedback survey page that collects anonymous course experience responses into a local CSV (data/course_feedback.csv) and exposes a lightweight summary widget (render_feedback_summary) for instructor/professor dashboards.

Engineering judgment: this module is evaluated against three professional criteria. (1) Operational correctness under Streamlit reruns (avoid repeated heavy loads, make side effects explicit): uses a st.form("course_feedback") so submission is atomic (no partial writes across reruns), reads the feedback file defensively (pd.read_csv inside try/except → empty DataFrame on failure), and appends new rows via _append_feedback() with an explicit “header only if file doesn’t exist” rule; the only side effect is a single CSV append on submit, and directory creation is idempotent (mkdir(parents=True, exist_ok=True)). (2) ML debuggability (log intermediate signals to localize failures: retrieval vs ranking vs synthesis): this module doesn’t instrument

retrieval/ranking/synthesis directly; instead it provides *product-level diagnostic signals* (satisfaction/engagement/project effectiveness + free-text notes) that can be correlated with RAG quality and UI changes, and it renders "Latest responses" to quickly spot shifts after model/index updates. (3) Product alignment (prioritize citations, timestamps, and 'why this result' cues to build user trust): it sets clear expectations ("Anonymous… stored locally"), captures an ISO timestamp per response, confirms successful submission, and provides an aggregated inspection (rates + latest rows) that supports continuous course/product improvement without exposing student identities.

Outcomes:
• validate the feedback CSV schema on load (required columns like timestamp, satisfied, engaging, project_effective, etc.), and if the file is malformed/unreadable, show a clear st.error() explaining how to fix it (e.g., "delete/rename corrupted course_feedback.csv to reset").
• define a single versioned schema for survey fields (allowed values for Yes/No radios, permitted preference options) and enforce it in _append_feedback() so dashboards don't break when labels/options change.
• cache _load_feedback_data() with st.cache_data keyed to the CSV file's mtime/size so reruns don't re-read the file repeatedly, especially when dashboards call render_feedback_summary() often.
• add a small "Feedback Storage Healthcheck" that verifies (a) feedback directory is writable, (b) CSV is readable, (c) required columns exist, and (d) a test row can be appended in-memory (dry-run), then reports PASS/FAIL with the precise failure cause.

smartta_unified/helpers.py

Role	Shared I/O and formatting utilities.
LOC	173
SHA1(10)	67a3c9048e
Docstring	Yes

Author intent: Shared utilities for SmartTA unified app that standardize logging + diagnostics payloads (NDJSON chat logs, feedback updates, dashboard-ready DataFrame loading), provide small UX helpers (AUB logo path lookup, question sanitization), and support Extensions interoperability (temporary cwd switch via ext_cwd()).

Engineering judgment: this module is evaluated against three professional criteria. (1) Operational correctness under Streamlit reruns (avoid repeated heavy loads, make side effects explicit): utilities are mostly pure transforms, with side effects isolated to explicit file operations (append_chat_log, update_feedback_entry) and a scoped working-directory context manager (ext_cwd) that reliably restores the previous cwd in finally. Log writes are deterministic (adds qid if missing, stamps UTC timestamp, normalizes contexts, and converts img_path into a boolean flag), which keeps reruns from producing inconsistent schemas. (2) ML debuggability (log intermediate signals to localize failures: retrieval vs ranking vs synthesis): it sanitizes and preserves the RAG debug payload in a stable, JSON-safe shape via _sanitize_debug (notably text_openai, img2img, text2img), and keeps retrieval evidence minimal-but-actionable in contexts (pdf, page, score) so

downstream UIs can compute top_score and display “why this result” diagnostics without drowning logs in raw text. (3) Product alignment (prioritize citations, timestamps, and ‘why this result’ cues to build user trust): it enforces consistent audit fields (qid, UTC timestamp), supports trustworthy reporting by producing a clean dashboard table (read_logs_df derives ts, date, top_score, and deduplicates by qid), and helps branded exports locate the correct AUB logo (aub_logo_path) while keeping potential sensitive fields lightweight (image attachment stored as presence/absence, not the raw path).

Outcomes:
• validate the log schema at write-time and read-time (required keys + types for qid/timestamp/contexts/debug), and in ext_cwd() assert config.SMARTTA_EXT_DIR exists/is a directory; otherwise raise a clear error (“Extensions directory missing—check install path / config wiring”).
• define a single CHAT_LOG_SCHEMA_VERSION + a canonical builder (e.g., make_chat_log_row(...)) so every writer produces the same keys (qid, timestamp, question, answer, contexts, debug, feedback, img_path) and dashboards don’t break when fields evolve.
• this module doesn’t load models, but it does re-parse logs; add a cached reader path (in the caller or via an optional wrapper) keyed by LOCAL_CHAT_LOG mtime/size so dashboards don’t rebuild the dataframe on every rerun for large NDJSON files.
• provide a helpers_healthcheck() that verifies (a) log directory writable, (b) NDJSON lines are valid JSON, (c) feedback update path is writable, and (d) ext_cwd() target exists—then returns a structured status dict that the UI can render as PASS/FAIL with the exact failing component.

(Extra “quiet but huge” fix aligned with your actual code paths: _write_log_line() currently rewrites the entire log file each append; switching to true append mode (open(path, "a")) + optional file lock would massively improve performance and reduce corruption risk as logs “”grow”.)

smartta_unified/style.py

Role	Styling utilities and UI coherence.
LOC	110
SHA1(10)	056cb64eeb
Docstring	Yes

Author intent: Static CSS for the SmartTA app, exported as a single BASE_STYLES string that the Streamlit entrypoint injects to enforce a consistent dark theme and polished UI components (title banner, chat bubbles/avatars, model badges, typing indicator, mode-selector radio “cards”, retrieval/citation pills, feedback buttons/tags, and tab styling).

Engineering judgment: this module is evaluated against three professional criteria. (1) Operational correctness under Streamlit reruns (avoid repeated heavy loads, make side effects explicit): it’s a pure constant (BASE_STYLES) with no runtime side effects—safe under Streamlit reruns and imports; styling behavior is explicit and only applies when the caller injects it via st.markdown(..., unsafe_allow_html=True). (2) ML

debuggability (log intermediate signals to localize failures: retrieval vs ranking vs synthesis): it does not log ML signals, but it supports ML debuggability in the UI by visually separating model types (.model-* badges), and by styling “retrieval evidence” components (.retrieval-row, .pdf-pill, .page-pill, .score-*) so debug outputs from the RAG pipeline are readable and inspectable. (3) Product alignment (prioritize citations, timestamps, and ‘why this result’ cues to build user trust): it emphasizes trust cues through clean hierarchy and consistent “evidence” visuals (citation pills + score pills + feedback state tags), and improves usability by making the mode selector look like a product-level navigation control rather than a default Streamlit radio.

Outcomes:
• Add invariant checks (index/metadata alignment, embedding dimensions) that fail loudly with actionable messages. For this module’s scope: add a lightweight UI “style sanity check” in the entrypoint (e.g., assert BASE_STYLES contains required selectors like .smartta-title, .chat-card, .retrieval-row); if missing/empty, show st.error("CSS not loaded: BASE_STYLES missing required blocks") to avoid silent regressions.
• Centralize structured logging schemas to prevent analytics drift. For this module’s scope: centralize and version the CSS tokens + class contract (e.g., document the expected class names used across chat.py/dashboard.py/youtube.py) so UI components don’t drift (.feedback-btn, .pdf-pill, .model-badge) and break styling after refactors.
• Cache heavy resources (models, indexes) to minimize Streamlit rerun latency. For this module’s scope: avoid repeated CSS injections by setting a session flag (e.g., st.session_state["styles_loaded"]) and injecting once per session—small win, but removes redundant work and reduces the chance of duplicated/stacked style blocks during complex reruns.
• run a known query, verify results, report status in UI. For this module’s scope: add a “UI Healthcheck” that renders one example widget of each styled element (mode radio, a chat bubble row, a retrieval pill row, feedback buttons) and reports PASS/FAIL if Streamlit DOM changes break the radio selectors (your CSS relies on div[role="radio"] which can be fragile across Streamlit versions).

5.3 RAG Package (smartta_unified/rag/)

smartta_unified/rag/settings.py

Role	Single source of truth for models/paths/constants.
LOC	27
SHA1(10)	813b415dee
Docstring	Yes

Author intent: Shared configuration for the SmartTA RAG engine that centralizes (a) project/path resolution for the legacy RAG data layout (SmartTA_RAG/data/index), (b) OpenAI client initialization + model IDs, and (c) low-level numerical/runtime constants (epsilon, chunk sizing, device selection).

Engineering judgment: this module is evaluated against three professional criteria. (1) Operational correctness under Streamlit reruns (avoid repeated heavy loads, make side

effects explicit): it performs only lightweight work at import time (loads .env, computes paths, selects device) but it does create an OpenAI client at import time, which is a side effect that can surface misconfiguration early (missing API key) and should be consciously managed under reruns/imports. (2) ML debuggability (log intermediate signals to localize failures: retrieval vs ranking vs synthesis): rather than logging pipeline signals, it enables debuggability by making the RAG engine's "knobs" explicit and centralized (embedding model ID, CLIP model ID, chunk limits, device choice), which reduces ambiguity when tracing retrieval and embedding behavior across environments. (3) Product alignment (prioritize citations, timestamps, and 'why this result' cues to build user trust): it supports trust indirectly by locking the engine to consistent model identifiers and stable chunk sizing, which helps keep retrieval/citation behavior consistent across runs and deployments.

Key dependencies:
• future.annotations
• dotenv.load_dotenv
• openai.OpenAI
• torch

Outcomes:
• validate OPENAI_API_KEY is present (or allow client=None until needed) and validate that INDEX_DIR exists/is readable when the engine starts (raise a clear error like "Index folder missing: expected SmartTA_RAG/data/index").
• define a small "engine config inspection" (models, device, chunk size, index dir) that downstream logging can attach to events—so analytics can always be traced back to the exact configuration used.
• keep this module as "settings only", and move OpenAI client creation into a cached factory (e.g., get_openai_client()), so reruns don't repeatedly re-initialize clients and so callers can swap keys/settings safely in tests.
• add a tiny config-level healthcheck_settings() (or a function in the engine layer that calls into settings) that verifies .env loaded, API key presence policy, device availability, and index path existence—then returns a PASS/FAIL status dict the UI can display.

smartta_unified/rag/state.py

Role	Mutable shared state for indexes and metadata; initialization.
LOC	22
SHA1(10)	2cd6637e07
Docstring	Yes

Author intent: Mutable shared state for FAISS indexes and metadata that acts as an in-process "singleton registry" for the RAG engine: it holds the loaded FAISS indexes (faiss_text, faiss_img, faiss_text_clip), the aligned corpora (chunks, chunk_meta, image_meta, page_to_chunk_ids), optional sparse-retrieval artifacts (bm25, X_text, X_img, X_text_clip), and the loaded CLIP objects (clip_model, clip_processor) so other RAG modules can read/write them without repeatedly loading resources.

Engineering judgment: this module is evaluated against three professional criteria. (1) Operational correctness under Streamlit reruns (avoid repeated heavy loads, make side effects explicit): keeping these as module-level globals is a pragmatic way to avoid re-loading large indexes/models on every rerun, but it also means state lifetime is "process-wide" and implicit—there's no explicit initialization/reset boundary here, so correctness depends on calling code to populate these consistently and in the right order. (2) ML debuggability (log intermediate signals to localize failures: retrieval vs ranking vs synthesis): centralizing the artifacts makes it easier for debugging tools/diagnostic UIs to inspect what's currently loaded (which index, which metadata, which CLIP model), but there are no built-in guards ensuring that chunks, chunk_meta, and FAISS index row counts stay aligned—so failures can show up later as confusing retrieval mismatches rather than an early, clear error. (3) Product alignment (prioritize citations, timestamps, and 'why this result' cues to build user trust): the presence of chunk_meta and page_to_chunk_ids directly supports traceable citations (pdf/page → chunk ids) and "why this result" displays, but only if upstream loaders maintain strict alignment and completeness across these structures.

Key dependencies:
• future.annotations

Outcomes:
• provide a validate_state() (called after loading) that asserts len(chunks) == len(chunk_meta) == faiss_text.ntotal (when set), verifies page_to_chunk_ids ids are within bounds, and checks that image_meta matches faiss_img.ntotal when image search is enabled—otherwise raise a clear exception explaining which loader/index is inconsistent.
• define a small "state inspection" dict helper (counts, which indexes are loaded, which features enabled) so logs/healthchecks can record consistent signals like ntotal, n_chunks, n_pages_mapped, and "clip_loaded" without each module inventing its own fields.
• wrap state ownership behind explicit get_or_load_*() functions in the loader module (or a dedicated state_manager) that populates these globals once and returns them, rather than letting multiple modules mutate globals directly (reduces accidental reloads and half-initialized states).
• add a state_healthcheck() that verifies "loaded vs expected" (indexes not None, corpora lengths consistent, CLIP objects present when multimodal search is enabled) and returns PASS/FAIL with the exact missing/misaligned component, so the UI can report "index loaded but metadata misaligned" instead of failing later during retrieval.

smartta_unified/rag/indexes.py

Role	Loads FAISS indexes; binds metadata; validates assumptions.
LOC	75
SHA1(10)	4f610395b3
Docstring	Yes

Author intent: Index loading helpers for SmartTA RAG that materialize the retrieval backbone by loading the text FAISS index + embeddings, the aligned chunk/metadata stores, the page→chunk mapping used for citations, and (when available) the CLIP image/text indexes, then building a BM25 sparse retriever over the loaded chunks.

Engineering judgment: this module is evaluated against three professional criteria. (1) Operational correctness under Streamlit reruns (avoid repeated heavy loads, make side effects explicit): load_indexes() is a heavy, side-effectful routine (disk IO + FAISS reads + numpy loads) that mutates global state in state; it partially protects correctness by explicitly clearing and repopulating state.chunks, state.chunk_meta, state.image_meta, and state.page_to_chunk_ids, but it does not guard against redundant reloads (it relies on the caller to only run it once per session). (2) ML debuggability (log intermediate signals to localize failures: retrieval vs ranking vs synthesis): it emits simple load-time signals (print of base folder and item counts) and makes CLIP coupling explicit via _clip_tag(settings.CLIP_MODEL_ID) (so image index selection tracks the active CLIP model), but it lacks validations that would catch the most common "retrieval is broken" causes (dimension mismatch, misaligned metadata lengths, wrong CLIP tag files missing). (3) Product alignment (prioritize citations, timestamps, and 'why this result' cues to build user trust): it loads and stores the exact artifacts required to justify answers—chunk_meta + page_to_chunk_ids for citations and image_meta for image evidence—yet it currently assumes these files are correct and aligned, which can silently degrade user trust if the mappings drift.

Public surface inspection:
• Functions: load_indexes

Key dependencies:
• future.annotations
• faiss
• pickle
• rank_bm25.BM25Okapi
• settings
• state
• json (reads *.jsonl)
• numpy (loads X_*.npy vectors)

Outcomes:
• after loading, assert state.faiss_text.ntotal == len(state.chunks) == len(state.chunk_meta) == state.X_text.shape[0], verify state.X_text.shape[1] == state.faiss_text.d, and validate page_to_chunk_ids contains only in-range chunk ids; for optional CLIP indexes check ntotal and vector shapes match their X_* arrays, and raise a clear error pointing to "rebuild indexes / regenerate embeddings for current CLIP tag".
• replace print(...) with a structured "index_load_summary" payload (base path, file presence, ntotal counts, d dims, CLIP tag, optional indexes present/missing) that can be reused by the UI healthcheck and dashboards.
• add an idempotency guard (early-return if already loaded and consistent), or introduce

load_indexes(force: bool = False); alternatively provide a cached factory at the caller layer (st.cache_resource) that calls load_indexes() once and returns a validated inspection, preventing unexpected reloads if session state resets.
• add a healthcheck_indexes() that (a) verifies required files exist (text.faiss, X_text.npy, chunks.jsonl, chunk_meta.jsonl, page_to_chunk_ids.pkl), (b) loads minimal headers/metadata, (c) runs the invariant checks above, and (d) reports PASS/FAIL + concrete remediation steps (e.g., "missing images_{tag}.faiss → run CLIP indexing with CLIP_MODEL_ID = …").

smartta_unified/rag/embeddings.py

Role	Embedding computation wrappers (text + image).
LOC	64
SHA1(10)	6e1b153de4
Docstring	Yes

Author intent: Embedding helpers for SmartTA RAG that provide a unified way to generate normalized vector embeddings for (a) text via OpenAI embeddings and (b) text/images via CLIP, while lazily loading the CLIP model/processor into shared state to avoid repeated heavy initialization.

Engineering judgment: this module is evaluated against three professional criteria. (1) Operational correctness under Streamlit reruns (avoid repeated heavy loads, make side effects explicit): CLIP is loaded lazily through _load_clip() and stored in state.clip_model / state.clip_processor, so repeated calls during reruns reuse the same loaded objects; inference functions are wrapped with @torch.no_grad() and move tensors to settings.device, making the "heavy" part explicit and inference-safe. (2) ML debuggability (log intermediate signals to localize failures: retrieval vs ranking vs synthesis): this module doesn't emit logs, but it makes embedding behavior reproducible by centralizing model IDs (settings.OPENAI_EMB_MODEL, settings.CLIP_MODEL_ID), enforcing L2 normalization for every embedding output (using settings.EPS for numerical stability), and returning consistent float32 numpy arrays—so downstream retrieval issues are easier to diagnose as "embedding config mismatch" vs "index mismatch." (3) Product alignment (prioritize citations, timestamps, and 'why this result' cues to build user trust): it supports trust indirectly by ensuring embeddings are stable, normalized, and consistent across modalities, which improves retrieval reliability and therefore the quality of citations/timestamps shown elsewhere (while keeping this module focused strictly on embedding generation).

Public surface inspection:
• Functions: embed_text_openai, embed_image_clip, embed_text_clip_batch

Key dependencies:
• PIL.Image
• future.annotations
• settings
• state
• torch
• transformers.CLIPModel
• transformers.CLIPProcessor

Outcomes:
• assert _load_clip() leaves state.clip_model and state.clip_processor non-None; validate OpenAI responses are non-empty and produce a 2D array; validate CLIP embeddings are finite (no NaNs) and have consistent dimensions across calls; if embed_image_clip receives a path, check file exists and is a valid image before attempting .open().
• add an optional lightweight "embedding event" record (model_id, device, batch_size, n_items, dim, elapsed_ms, error_type) so embedding failures/slowdowns can be traced consistently without each caller inventing its own fields.
• _load_clip() already memoizes via state, but make it more robust by also guarding state.clip_processor (not just clip_model) and optionally exposing a cached factory (e.g., in the Streamlit layer via st.cache_resource) to avoid reloads if the module gets re-imported in dev.
• add a healthcheck_embeddings() that (a) embeds a known short text via OpenAI, (b) embeds the same text via CLIP batch, (c) optionally embeds a tiny in-memory RGB image via CLIP, then verifies shapes, finite values, and unit-norm—returning PASS/FAIL with the exact failing step and model id.

smartta_unified/rag/search.py

Role	Multi-channel retrieval logic and scoring.
LOC	85
SHA1(10)	af7b026c26
Docstring	Yes

Author intent: Search utilities for SmartTA RAG that run lightweight retrieval over already-loaded global indexes in state: (1) a fused OpenAI dense + BM25 text search returning chunk-level scores, and (2) CLIP-driven retrieval paths that either boost PDF pages from an attached screenshot or retrieve chunks via CLIP text/image → faiss_text_clip.

Engineering judgment: this module is evaluated against three professional criteria. (1) Operational correctness under Streamlit reruns (avoid repeated heavy loads, make side effects explicit): all functions are side-effect free and only depend on pre-loaded state objects; they guard "feature not available" cases explicitly (e.g., faiss_img is None, faiss_text_clip is None/empty, missing query/image) by returning {} rather than crashing. The only "heavy" work is embedding calls + FAISS searches, which are explicit in each function and rely on upstream caching/state (this module does not load indexes/models itself). (2) ML debuggability (log intermediate signals to localize failures: retrieval vs

ranking vs synthesis): retrieval behavior is transparent and deterministic—search_text_only() min-max normalizes dense FAISS scores and BM25 scores separately, then fuses them with fixed weights 0.6 (dense) / 0.4 (keyword); search_image_boost() applies an explicit similarity floor (MIN_SIM = 0.25) and type-aware weighting (W_EMBED vs W_PAGE) before normalizing. However, there is no structured debug payload emitted (no return of raw vs normalized scores, no warnings when required state is missing), which can make "empty results" ambiguous (missing indexes vs low similarity vs embed failure). (3) Product alignment (prioritize citations, timestamps, and 'why this result' cues to build user trust): functions return normalized, comparable scores in [0,1] and preserve explainable keys—chunk ids for text/chunk retrieval and (pdf, page) keys for screenshot boosting—so downstream UI can show "why this result" evidence (top pages/chunks, scores) and produce reliable citations.

Public surface inspection:
• Functions: search_text_only, search_image_boost, search_chunks_from_text_clip, search_chunks_from_image_clip

Key dependencies:
• future.annotations
• embeddings.embed_text_openai
• embeddings.embed_image_clip
• embeddings.embed_text_clip_batch
• state (expects faiss_text, bm25, faiss_img, image_meta, faiss_text_clip to be populated)
• numpy (normalization + argsort)

Outcomes:
• before searching, assert required state exists (state.faiss_text and state.bm25 for search_text_only; state.faiss_img + state.image_meta for search_image_boost; state.faiss_text_clip for CLIP chunk search). If missing, return a clear error object (or raise a targeted exception caught by UI) stating "indexes not loaded—run load_indexes() / check engine_loaded".
• return (or optionally log) a compact debug dict per search including mode, k, min_sim, weights, n_candidates_before/after_threshold, and top hits—so dashboards can consistently compare retrieval behavior across releases.
• keep this module pure, but add an optional cache_key strategy in the caller (or an optional @st.cache_data wrapper) for repeated normalization/fusion on identical queries in the same session; biggest win is still ensuring embeddings and indexes are cached upstream.
• a search_healthcheck() that runs (a) a fixed text query through search_text_only, (b) a CLIP text query through search_chunks_from_text_clip (if available), and (c) optional screenshot boost on a known test image—then validates non-empty outputs and score ranges, reporting PASS/FAIL with the exact missing component or failure reason.

smartta_unified/rag/engine.py

Role	End-to-end retrieve → synthesize → log pipeline.

LOC	254
SHA1(10)	2fac859658
Docstring	Yes

Author intent: High level answer engine for SmartTA that (1) fuses multiple retrieval signals (OpenAI dense text, BM25, CLIP text↔chunk, CLIP text↔image→page projection, screenshot→page boosting, screenshot→CLIP chunk search) into a single ranked chunk list (hybrid_search_v3), then (2) synthesizes a final answer via OpenAI Chat Completions using only the formatted context and returning citations + debug score traces (answer_with_citations).

Engineering judgment: this module is evaluated against three professional criteria. (1) Operational correctness under Streamlit reruns (avoid repeated heavy loads, make side effects explicit): all "heavy" operations are explicit and delegated—retrieval assumes indexes/models are already loaded into state (no index loading here), while answer_with_citations performs the single expensive LLM call (settings.client.chat.completions.create) and optionally computes extra debug retrieval traces (image boost + OpenAI embedding probe). It is mostly side-effect free except for network calls and print()-based warnings. (2) ML debuggability (log intermediate signals to localize failures: retrieval vs ranking vs synthesis): it returns retrieval evidence directly—contexts (materialized chunks with (pdf,page,score)), plus page_scores split into img2img (screenshot→page boost), text2img (question→image index→page scores), and text_openai (OpenAI embedding probe against faiss_text). Hybrid fusion is also inspectable in code via explicit weights (w_base, w_t2t, w_i2t, w_t2p, w_i2p) including an "image-focused" heuristic for short/visual questions. (3) Product alignment (prioritize citations, timestamps, and 'why this result' cues to build user trust): the system prompt enforces "ONLY the provided context" and inline citations [pdf:page]; context formatting truncates long chunks to settings.MAX_CHARS_PER_CHUNK to keep prompts stable; the engine can optionally include a screenshot as a data:<mime>;base64,... image input so the model can ground answers when students ask "what's in this figure".

• Functions: hybrid_search_v3, answer_with_citations

Key dependencies:
• future.annotations
• base64
• embeddings.embed_text_clip_batch
• embeddings.embed_text_openai
• search._minmax_norm
• search.search_chunks_from_image_clip
• search.search_chunks_from_text_clip
• search.search_image_boost
• search.search_text_only
• settings
• state

Outcomes:
• before searching, assert required state is loaded (e.g., state.faiss_text, state.bm25, state.chunks/chunk_meta, and when used state.faiss_img/image_meta, state.faiss_text_clip, state.page_to_chunk_ids); if missing, raise/return a clear “Indexes not loaded—call load_indexes() / engine_loaded is False” error instead of silently returning empty hits.
• replace print() warnings with a structured debug dict that includes (a) chosen fusion weights, (b) which retrieval channels were active, (c) candidate counts, and (d) top hits—so dashboards/UI can compare retrieval behavior across versions consistently.
• ensure upstream callers cache/index-load once (this module assumes it), and make debug truly opt-in—right now answer_with_citations still computes page_scores_* and the OpenAI embedding probe even when debug=False (it only suppresses the warning print).
• add an engine_healthcheck() that runs a fixed query through hybrid_search_v3 (expect non-empty hits + valid (pdf,page)), then runs answer_with_citations with a tiny context budget and verifies citation format—returning PASS/FAIL with the failing subsystem (missing indexes vs embedding failure vs LLM call).

(Quick code-accuracy note worth fixing: in the non-screenshot branch, your user message currently uses "Context...{ctx}" instead of "Context:\n{ctx}", which weakens grounding. That’s a high-impact, low-risk correction inside answer_with_citations.)

5.4 Lecture Search Subsystem (SmartTA_Extensions/)

SmartTA_Extensions/backend/resources.py

Role	Singleton model/resource loader; performance critical.
LOC	154
SHA1(10)	c4daa6f67f
Docstring	Yes

Author intent: Centralized resource management for SmartTA models and indices. Provides singleton-style access to: - Sentence transformer embedding model - Cross-encoder reranking model - FAISS search index - Metadata cache. Reduces redundant loads and improves memory footprint by caching loaded objects in module-level globals and reusing them across calls (including Streamlit reruns within the same process).

Engineering judgment: this module is evaluated against three professional criteria. (1) Operational correctness under Streamlit reruns (avoid repeated heavy loads, make side effects explicit). Heavy loads are triggered only on demand (first call to each getter), then reused via _sentence_transformer, _cross_encoder, _index, _metadata. get_index_and_metadata() is idempotent for the same index_path unless force_reload=True, and clear_cache() provides an explicit reset boundary. (2) ML debuggability (log intermediate signals to localize failures: retrieval vs ranking vs synthesis). It logs all major load events and failures, returns None instead of crashing on model/index load errors, and includes a targeted “index may be corrupted” tip when faiss.read_index() fails; get_cache_status() exposes what is loaded plus basic counts

(ntotal, metadata length) for diagnostics. (3) Product alignment (prioritize citations, timestamps, and 'why this result' cues to build user trust). While it doesn't handle citations directly, it improves user experience by keeping retrieval/reranking resources consistent and avoiding repeated loads that would cause slow or inconsistent behavior during interactive use.

Public surface inspection:
• Functions: get_sentence_transformer, get_cross_encoder, get_index_and_metadata, clear_cache, get_cache_status

Key dependencies:
• faiss
• logging
• sentence_transformers.CrossEncoder
• sentence_transformers.SentenceTransformer
• plus stdlib: os, json, typing (path checks + metadata IO + typing)

Outcomes:
• after loading, validate len(metadata) == index.ntotal (or at least >= ntotal depending on your schema), validate metadata is a list of dicts with required keys, and (optionally) run a tiny FAISS sanity query (index.search(zeros, k=1)) to catch corrupted/shape-mismatch indexes early.
• emit consistent structured logs (event name + fields like model_name, index_path, ntotal, metadata_len, force_reload, error_type) so downstream dashboards/alerts can reliably parse behavior.
• keep the singleton approach, but make caching safer by keying the cache on both index_path and metadata_path (right now the "already loaded" short-circuit only checks index_path, which can accidentally serve stale metadata).
• add a healthcheck_resources(index_path, metadata_path) that loads resources (without mutating cache if desired), validates invariants, and returns a structured PASS/FAIL report (missing file vs load error vs alignment mismatch) for the UI/admin tools.

SmartTA_Extensions/backend/indexing.py

Role	Transcript indexing and persistence.
LOC	162
SHA1(10)	16f5c77e33
Docstring	Yes

Author intent: FAISS Index Management and Embeddings. Handles incremental vector indexing for the YouTube lecture segments pipeline by (a) providing Streamlit-cached accessors for the embedding/reranking models (delegating to backend.resources), (b) loading or creating a FAISS IVF index when needed, and (c) embedding and appending only new lectures into data/course.index plus aligned JSON metadata in data/course_meta.json.

Engineering judgment: this module is evaluated against three professional criteria. (1) Operational correctness under Streamlit reruns (avoid repeated heavy loads, make side effects explicit). The heavy models are cached with @st.cache_resource (load_sentence_transformer, load_cross_encoder) and all other heavy work is explicit inside reindex_if_needed() (FAISS IO, embedding generation, index mutation, disk persistence). It is rerun-safe in the sense that it detects already-indexed lectures via persisted META_PATH and returns early when there are no new lectures; however, it still performs significant side effects when new lectures exist (progress UI, index.add, faiss.write_index, JSON rewrite). (2) ML debuggability (log intermediate signals to localize failures: retrieval vs ranking vs synthesis). It logs and surfaces index-load failures clearly (logger + Streamlit warnings) and exposes a practical "index corruption" remediation tip; it also makes the indexing strategy explicit (IVF-PQ for >10k segments, IVF-Flat otherwise, with NPROBE tuning). What it does *not* do is validate embedding dimensions/dtypes or confirm metadata↔index alignment after updates, which can turn "retrieval is weird" into a late failure downstream. (3) Product alignment (prioritize citations, timestamps, and 'why this result' cues to build user trust). It enriches metadata with start/end timestamps and attempts to attach category (via load_youtube_lectures_func) to support better organization in the UI; it also provides user-facing progress + ETA and a success toast, which improves perceived reliability during indexing.

Public surface inspection:
• Functions: load_sentence_transformer, load_cross_encoder, get_index_hash, reindex_if_needed

Key dependencies:
• backend.resources.get_cross_encoder
• backend.resources.get_sentence_transformer
• faiss
• logging
• streamlit
• numpy
• utils.config.DEVICE (declared, not used directly in this file)
• utils.config.EMBED_DIM
• utils.config.INDEX_PATH
• utils.config.META_PATH
• utils.config.MODEL_DIR (declared, not used directly in this file)
• utils.config.NPROBE

Outcomes:
• before index.add, assert embeddings.shape[1] == EMBED_DIM and embeddings.dtype == np.float32 (cast if needed); after indexing, assert index.ntotal == len(metadata) (or track a separate row-count) and show a clear st.error() if misaligned ("metadata/index mismatch — delete META_PATH/INDEX_PATH and rebuild").
• replace ad-hoc warnings and UI messages with a single structured event payload (e.g., index_load_failed, index_created, lectures_indexed, segments_added, ntotal_after,

nprobe, index_type) so dashboards/debugging can reliably compare runs.
• keep @st.cache_resource for models, but also consider caching a "loaded index + metadata" inspection keyed by get_index_hash() so reruns don't repeatedly faiss.read_index + json.load when nothing changed (and add a force_reload option for admin actions).
• add healthcheck_index() that verifies INDEX_PATH/META_PATH readability, confirms IVF indexes are trained (index.is_trained), checks nprobe is applied, validates metadata schema keys (lecture/text/start/end/category), and runs a tiny sanity search on a dummy vector—returning PASS/FAIL with the exact failing step and a suggested fix.

SmartTA_Extensions/backend/search.py

Role	Retrieval + reranking; timestamps and relevance.
LOC	690
SHA1(10)	ad15998acb
Docstring	Yes

Author intent: Search and Ranking Functions that power the SmartTA_Extensions YouTube lecture search pipeline: correct common typos, run FAISS-based semantic retrieval over transcript segments, apply phrase/token heuristics to boost exact matches, expand queries with ML-aware synonyms/acronyms, deduplicate near-duplicate segments, and optionally rerank results using BM25-style scoring + a cross-encoder.

Engineering judgment: this module is evaluated against three professional criteria. (1) Operational correctness under Streamlit reruns (avoid repeated heavy loads, make side effects explicit). The core routines are side-effect free (they operate on passed metadata/index/embedder) except for explicit caching: _prepare_bm25() stores precomputed BM25 artifacts in st.session_state keyed by a quick metadata hash, and typo correction keeps a process-wide cached _CORPUS_VOCAB. The "heavy work" is explicit and localized to embedding calls (embedder.encode(...)) and scoring loops; the module does not load models/indexes itself. (2) ML debuggability (log intermediate signals to localize failures: retrieval vs ranking vs synthesis). It exposes interpretable retrieval signals by enriching segment dicts with similarity, distance, phrase_bonus, and final_score in search_segments(), plus base_score/phrase_bonus in hybrid_rank() and ce_score/final_score in precision_rerank(). However, some debuggability is weakened by silent fallbacks (e.g., if embedding fails, deduplication drops to token overlap only; BM25 uses a cached corpus without validating schema) and by a couple of unused parameters/imports (index_hash, hashlib) that suggest planned caching not currently active. (3) Product alignment (prioritize citations, timestamps, and 'why this result' cues to build user trust). The ranking strategy explicitly boosts exact phrases, bigrams/trigrams, and token overlap, and keeps segment metadata (lecture, start, end, text) intact—supporting trustworthy UI cues like "why this result," lecture grouping, and timestamp navigation while avoiding opaque "black box" scoring.

Public surface inspection:
• Functions: levenshtein_distance, correct_typos, search_segments,

deduplicate_segments, detect_question_type, expand_query_semantically, token_overlap_score, rerank_results, bm25_score, hybrid_rank, precision_rerank

Key dependencies:
• hashlib (imported; currently not essential to runtime behavior)
• logging
• streamlit (session_state cache for BM25 artifacts)
• numpy (cosine similarity + normalization)
• re, math (tokenization + BM25 math)

Outcomes:
• validate metadata entries contain required keys (text, lecture, start, end) before scoring/dedup; validate that _index.search(...) returns indices within bounds; validate embedder outputs are finite and correct shape; if not, raise/log a targeted message ("metadata schema invalid" vs "index mismatch" vs "embedder failure").
• emit a consistent debug record per query (question type, corrected query, expansion applied, weights used, candidate counts, top scores) rather than ad-hoc logger.warning(...), so UI + analytics can reliably compare versions.
• keep this module pure, but make caching hooks real—either use the passed index_hash to memoize candidate scoring, or remove it; also avoid metadata.index(c) inside hybrid_rank() by storing the original index id in each candidate at retrieval time (current approach can be $O(n^2)$ on large corpora).
• add a search_healthcheck() that runs a known query through search_segments → deduplicate → hybrid_rank → precision_rerank (if cross-encoder is available), verifies non-empty outputs and monotonic score ordering, and reports PASS/FAIL with the exact failing stage (typo vocab build vs FAISS search vs BM25 cache vs rerank).

SmartTA_Extensions/backend/transcription.py

Role	Transcription pipeline; I/O and preprocessing.
LOC	250
SHA1(10)	5688d549cc
Docstring	Yes

Author intent: Transcription Module that downloads/loads speech-to-text resources and generates timestamped lecture transcripts for the SmartTA_Extensions pipeline, writing per-lecture .txt transcripts into TRANSCRIPT_DIR and maintaining a consolidated segments JSON store in META_SEGMENTS (copied from app.py with the same logic and retry behavior).

Engineering judgment: this module is evaluated against three professional criteria. (1) Operational correctness under Streamlit reruns (avoid repeated heavy loads, make side effects explicit). Heavy work is gated and explicit: Whisper is loaded once via @st.cache_resource (load_whisper_model) and transcription only runs for lectures not already represented in META_SEGMENTS or existing .txt transcripts. Side effects are concentrated in transcribe_new_videos() (file IO writes to TRANSCRIPT_DIR and

META_SEGMENTS) and download_youtube_audio() (network + ffmpeg postprocessing via yt_dlp), with retry/backoff logic to reduce transient failures. (2) ML debuggability (log intermediate signals to localize failures: retrieval vs ranking vs synthesis). This module focuses on ingestion quality: it validates Whisper outputs by requiring result["segments"], filters malformed segments (missing text/start/end), and fails with clear errors when no valid segments are produced. It also surfaces actionable failure messages in the UI (install Whisper guidance; "common fixes" on transcription failure) and logs exceptions to help differentiate "model not available" vs "video not found" vs "transcription output invalid." (3) Product alignment (prioritize citations, timestamps, and 'why this result' cues to build user trust). It produces structured timestamped segments (start/end/text) that directly enable trustworthy downstream UX (jump-to-time playback, "why this hit" context), and it preserves deterministic outputs per lecture by writing both human-readable transcript text and machine-readable segments metadata.

Public surface inspection:
• Functions: download_youtube_audio, load_whisper_model, safe_operation, transcribe_new_videos

Key dependencies:
• backend.video.get_video_duration
• logging
• random
• streamlit
• utils.config.DATA_DIR
• utils.config.LECTURE_DIR
• utils.config.MAX_RETRIES
• utils.config.META_SEGMENTS
• utils.config.MODEL_DIR
• utils.config.RETRY_DELAY
• utils.config.TRANSCRIPT_DIR
• utils.config.WHISPER_MODEL_SIZE
• utils.helpers.get_all_lectures
• utils.helpers.process_with_progress
• yt_dlp

Outcomes:
• validate filesystem invariants before work starts (LECTURE_DIR/TRANSCRIPT_DIR exist & writable; META_SEGMENTS is valid JSON dict), and validate transcription invariants after each lecture (segments non-empty, start < end, name uniqueness) so failures don't silently degrade into partial metadata.
• emit consistent events like transcription_start/end, lecture_transcribed, lecture_skipped_existing, transcription_failed, whisper_load_failed, each with fields (lecture, duration, n_segments, error_type) so the professor dashboard can trend ingestion health over time.
• Whisper is already cached; extend caching to "already-seen transcripts" by caching the parsed META_SEGMENTS + transcript file list (keyed by mtime) to avoid repeated

directory scans and JSON reads on every rerun.
• add a transcription_healthcheck() that verifies Whisper import/load, confirms ffmpeg/yt-dlp pipeline availability (for downloads), checks read/write permissions for MODEL_DIR/TRANSCRIPT_DIR, and dry-runs model.transcribe on a tiny local test clip (if present) to report PASS/FAIL with the exact failing component.

SmartTA_Extensions/backend/analytics.py

Role	Analytics aggregation for instructor dashboard.
LOC	459
SHA1(10)	83cc5c550c
Docstring	Yes

Author intent: Analytics Module that logs YouTube-library search activity to a local JSON log (LOG_PATH), loads it into a DataFrame (cached 60s), and renders instructor-facing visualizations to understand what students search for, when they search, which lectures/time-points are "hot," and what learning progressions/repeated questions emerge over time. Copied from app.py without modifications.

Engineering judgment: this module is evaluated against three professional criteria. (1) Operational correctness under Streamlit reruns (avoid repeated heavy loads, make side effects explicit). Side effects are isolated to log_query() (reads existing JSON list → appends entry → rewrites file). Read-path is rerun-safe via @st.cache_data(ttl=60) on load_analytics_data(), reducing repeated disk IO. Visualization functions are pure with respect to storage and compute their aggregates deterministically from the provided DataFrame; the only persistent UI state is the timeline "carousel" index stored in st.session_state.timeline_graph_index. (2) ML debuggability (log intermediate signals to localize failures: retrieval vs ranking vs synthesis). While it does not instrument model internals directly, it logs and analyzes retrieval-facing evidence (results include lecture, start, end, and a 200-char snippet), enabling practical debugging signals like "hot timestamps" per lecture, repeated questions, and query patterns that often correlate with retrieval gaps or unclear lecture segments. (3) Product alignment (prioritize citations, timestamps, and 'why this result' cues to build user trust). The module's analytics are explicitly time-anchored (start/end → midpoint hotspot histograms) and lecture-anchored (frequency charts + engagement breakdown), producing actionable instructor views that map cleanly back to the lecture timeline students actually navigate. It also upgrades UX when Plotly is available (interactive charts + styling), with Streamlit fallbacks when not.

Public surface inspection:
• Functions: log_query, load_analytics_data, render_lecture_frequency_chart, render_query_timeline_analysis, render_query_patterns_over_time, render_student_engagement_analysis, render_learning_path_analysis

Key dependencies:
• collections.Counter
• logging

• streamlit
• pandas
• os, json, datetime, re
• utils.config.LOG_PATH (and imported DATA_DIR, currently unused)
• Optional: plotly.graph_objects (interactive charts, otherwise Streamlit fallbacks)

Outcomes:
• validate the log schema at load time (ensure results is a list of dicts with lecture/start/end), coerce/validate timestamps once (and warn on rows that become NaT), and surface a clear UI error when the log file is malformed instead of silently returning an empty DataFrame that looks like "no usage."
• define a versioned log-entry schema (field names + types + allowed values) and enforce it inside log_query() (e.g., stable qid, UTC timestamp, consistent lecture_filter encoding) so dashboards don't break when upstream modules evolve.
• keep load_analytics_data() cached, and also cache expensive derived aggregates (lecture counts, hourly counts, hotspot histograms) keyed by (LOG_PATH mtime, filters); additionally, switch log_query() from "rewrite whole JSON list" to JSONL append to avoid quadratic slowdowns and reduce corruption risk as logs grow.
• add an "Analytics Healthcheck" that verifies (a) LOG_PATH readable/writable, (b) JSON parses to a list, (c) required columns present, (d) timestamps parse cleanly, and (e) at least one visualization aggregate can be computed—reporting PASS/FAIL with the exact failing step.

SmartTA_Extensions/frontend/components.py

Role	UI primitives and consistent rendering.
LOC	244
SHA1(10)	5cc3524993
Docstring	No

Engineering judgment: this module is evaluated against three professional criteria. (1) Operational correctness under Streamlit reruns (avoid repeated heavy loads, make side effects explicit). It is primarily UI-rendering and is rerun-safe: render_search_results() and render_course_materials() are pure aside from explicit st.session_state updates when a user clicks "Jump to Timestamp" (sets jump_to + last_clicked_result then st.rerun()), and it avoids heavy model/index work entirely. It keeps expensive work bounded by design (e.g., truncates displayed segment text to [:600], only shows a small lecture distribution preview, and expands only the first lecture by default). (2) ML debuggability (log intermediate signals to localize failures: retrieval vs ranking vs synthesis). It surfaces retrieval quality cues directly in the UI by displaying per-segment score badges (score and/or kw_overlap) with color thresholds and preserves the exact lecture + start/end timestamps, but it does not expose deeper pipeline signals (e.g., semantic vs keyword contribution, reranker score) beyond what is already present in each result dict. (3) Product alignment (prioritize citations, timestamps, and 'why this result' cues to build user trust). It groups results by lecture, shows lecture categories and optional professor_note pulled from load_youtube_lectures(), highlights query terms in the

segment snippet, and provides an explicit “Jump to Timestamp” action that ties results back to the video timeline—strong “why this result / where in the lecture” trust cues.

Public surface inspection:
• Functions: render_app_header, render_search_results, render_course_materials

Key dependencies:
• streamlit
• utils.helpers.load_youtube_lectures
• os (file existence + course material downloads)
• re (query-term highlighting)
• time, datetime (timestamp formatting + click freshness)

Outcomes:
• validate result schema before rendering (lecture, start, end, text presence; numeric start/end; non-negative duration). If malformed, show st.error() indicating which field is missing and where it likely came from (search/rerank stage).
• emit a consistent UI event record (e.g., result_clicked, lecture_expanded, materials_downloaded) including lecture, start, end, score/kw_overlap, and query_text so frontend engagement can be correlated with backend retrieval quality using a stable schema.
• cache load_youtube_lectures() output (or at least the derived title → (category, professor_note) mapping and course_materials list) in st.session_state to avoid repeated disk/JSON reads and repeated category scans across reruns.
• add a lightweight “UI Data Healthcheck” that verifies load_youtube_lectures() returns expected keys (lectures, course_materials), checks that referenced course material files exist under uploads_dir, and validates that rendering a small synthetic result list works—reporting PASS/FAIL with precise missing fields/files.

(Extra critical fix for this module’s current implementation: because it uses unsafe_allow_html=True and injects query_text, raw_text, and professor_note into HTML, you should HTML-escape these strings before rendering to prevent broken markup and inadvertent HTML injection.)

SmartTA_Extensions/frontend/tabs/student.py

Role	Student UI workflow.
LOC	515
SHA1(10)	f3b51e4ca0
Docstring	No

Engineering judgment: this module is evaluated against three professional criteria. (1) Operational correctness under Streamlit reruns (avoid repeated heavy loads, make side effects explicit). The tab is mostly rerun-safe UI orchestration: it initializes st.session_state keys for search results + click state, clears jump_to when the selected lecture changes, and uses explicit st.rerun() when the user “Back to selected lecture.” Heavy work is explicit and delegated—indexing is expected to be done upstream (it relies

on provided metadata/index), while search work is performed on submit (embedding + FAISS search + reranking) and results are cached in st.session_state["query_cache"] keyed by query|mode|idx_hash. Side effects are limited to analytics logging (log_query(...)) and embedding an iframe with unsafe_allow_html=True. (2) ML debuggability (log intermediate signals to localize failures: retrieval vs ranking vs synthesis). The module exposes several retrieval-stage signals in the UI flow: it runs detect_question_type() + expand_query_semantically(), then either hybrid ranks results (BM25 + embeddings) and deduplicates by semantic similarity/time overlap. It also records "what happened" through progress/status placeholders and stores the final segments list in session state. However, it does not surface (or persist) debug traces of *why* a segment scored highly beyond what is embedded in each result dict, and the "precision" (cross-encoder) branch is effectively unreachable because search_mode is hard-set to "🎯 Quality." (3) Product alignment (prioritize citations, timestamps, and 'why this result' cues to build user trust). It strongly anchors results to the learning artifact: lecture selection by category, a YouTube preview pane with jump-to-timestamp behavior, and post-search UX ("Search Complete" toast, grouped result rendering via render_search_results, and a follow-up "Ask AI" summarizer that cites segments as [1] [2] and links back to youtu.be?t= timestamps).

Public surface inspection:
• Functions: render_student_tab

Key dependencies:
• backend.analytics.log_query
• backend.indexing.get_index_hash
• backend.indexing.load_cross_encoder
• backend.indexing.load_sentence_transformer
• backend.indexing.reindex_if_needed
• backend.search.deduplicate_segments
• backend.search.detect_question_type
• backend.search.expand_query_semantically
• backend.search.hybrid_rank
• backend.search.precision_rerank
• backend.search.search_segments
• backend.search.token_overlap_score
• frontend.components.render_course_materials
• frontend.components.render_search_results
• frontend.styles.apply_search_ui_styles
• frontend.styles.render_search_header
• streamlit
• utils.helpers.get_video_id_from_title
• utils.helpers.get_youtube_lectures_categorized *(imported but not used in this file)*
• utils.helpers.load_youtube_lectures

Outcomes:
• before running search, validate _metadata is a non-empty list and _index is not None; if

missing, explain that the tab expects preloaded index/metadata (and call out the intended fallback). After retrieval, validate each result has lecture/start/end/text and start <= end before rendering/jump-to.
• extend log_query(...) payload to include mode (quality/fast/precision), selected_lecture and only_this_lecture, and whether the answer came from cache. Right now it logs lecture_filter=None unconditionally, which loses crucial segmentation.
• the query cache is good, but the cache key should include the lecture filter when "Current lecture only" is enabled (otherwise cache hits can return broad results that get filtered down badly). Also, fix the fallback broad-search path to actually update st.session_state["last_search_results"] (currently it rebinds a local results variable only).
• add a "Student Tab Healthcheck" button that verifies (a) index+metadata loaded, (b) get_index_hash() works, (c) one dry-run search_segments() returns valid indices, and (d) OpenAI follow-up prerequisites are met (OPENAI_API_KEY + openai installed) — then reports PASS/FAIL with the exact failing step.

SmartTA_Extensions/frontend/tabs/professor.py

Role	Professor UI workflow (analytics).
LOC	728
SHA1(10)	2f0519c947
Docstring	No

Engineering judgment: this module is evaluated against three professional criteria. (1) Operational correctness under Streamlit reruns (avoid repeated heavy loads, make side effects explicit). render_professor_tab(...) is a large UI orchestrator with explicit side effects (writing/deleting files, mutating youtube_lectures JSON, updating META_SEGMENTS / META_PATH, and writing INDEX_PATH). It keeps reruns mostly safe by gating destructive actions behind confirmation flags in st.session_state and using forms for multi-field edits, but it also mixes "admin workflows" (Add+Transcribe+Index) directly into the UI path, so partial failures can leave the system in an inconsistent intermediate state (e.g., lecture saved but audio download/transcription fails). (2) ML debuggability (log intermediate signals to localize failures: retrieval vs ranking vs synthesis). It helps debug system behavior at the product/usage layer: "Top Confused Topics" mines LOG_PATH and shows representative questions plus the top matching lecture timestamps, while "Clean Index" can rebuild the index from metadata to recover from deletions. However, it does not produce structured diagnostics for the ingestion pipeline (download → whisper → index add), and it hard-codes index rebuild behavior (rebuilds as IndexFlatL2) which can change retrieval performance without clear reporting. (3) Product alignment (prioritize citations, timestamps, and 'why this result' cues to build user trust). The professor UX is strongly aligned: it supports adding lectures, attaching professor notes, organizing downloadable course materials into ordered sections with per-file descriptions, managing existing lectures, and viewing analytics (lecture-frequency + timeline + query patterns) that directly map back to timestamps students can revisit.

Public surface inspection:
• Functions: render_professor_tab

Key dependencies:
• backend.analytics.load_analytics_data
• backend.analytics.render_lecture_frequency_chart
• backend.analytics.render_query_patterns_over_time
• backend.analytics.render_query_timeline_analysis
• backend.indexing.load_sentence_transformer
• backend.transcription.download_youtube_audio
• backend.transcription.load_whisper_model
• faiss
• logging
• streamlit
• utils.helpers.delete_lecture_file *(used as a failure cleanup, but doesn't fully rollback the lecture entry)*
• utils.helpers.extract_youtube_id
• utils.helpers.get_all_lectures *(imported as helpers_get_all_lectures but unused in this file)*
• utils.helpers.load_youtube_lectures
• utils.helpers.save_youtube_lectures

Outcomes:
• after any indexing operation, assert index.ntotal == len(meta) and that embeddings are float32 with the same dimension as the index; also validate transcription output (non-empty segments, start < end) before writing META_SEGMENTS.
• standardize event records for admin actions (lecture_added, transcription_failed, index_updated, materials_section_deleted) with consistent fields (lecture_id, title, n_segments, error_type), instead of scattered st.error(...) + logger.error(...) strings.
• Whisper is already cached in load_whisper_model(), but index/metadata reloads inside this tab can be cached (read META_PATH/INDEX_PATH once per run) and "Clean Index" should report what index type it rebuilt to (and ideally preserve IVF settings rather than always rebuilding as IndexFlatL2).
• run a known query, verify results, report status in UI: add a "System Healthcheck" button that verifies paths exist/writable, Whisper loads, FAISS index reads, META_PATH parses, and a tiny dry-run embedding + index.search() succeeds—then reports PASS/FAIL + a concrete fix (rebuild index, clear corrupted JSON, etc.).

(Concrete bugs worth fixing in this file: the "Cancel" button in the lecture edit form calls st.rerun without parentheses, and renaming a lecture updates transcripts/segments but does not update META_PATH entries or rebuild the FAISS index to reflect the new lecture title.)

SmartTA_Extensions/utils/config.py

Role	Central configuration constants for Extensions.
LOC	76
SHA1(10)	2b8dafcad6
Docstring	Yes

Author intent: Central configuration constants for SmartTA. All path, device, retry, and search parameters centralized here while preserving existing values; additionally, it creates the shared app data directory (PROJECT_ROOT/data) and migrates the legacy Extensions log (SmartTA_Extensions/data/queries_log.json) into the unified log location (PROJECT_ROOT/data/youtube_queries_log.json) when needed.

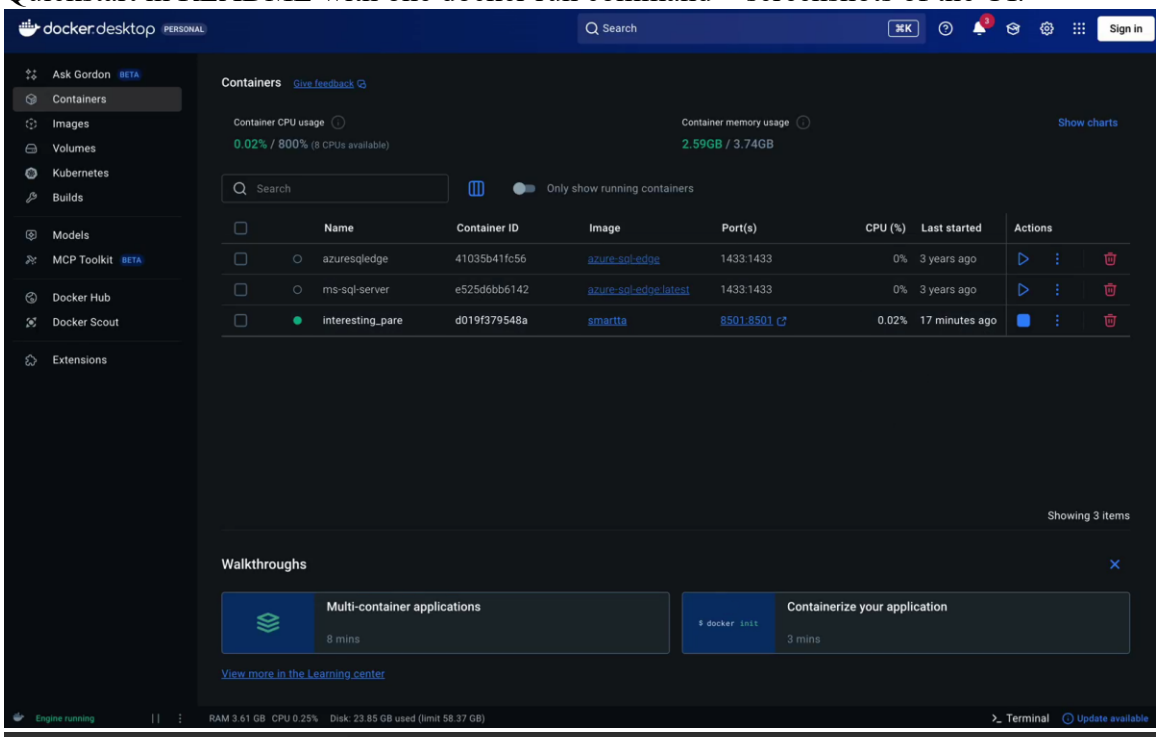
Engineering judgment: this module is evaluated against three professional criteria. (1) Operational correctness under Streamlit reruns (avoid repeated heavy loads, make side effects explicit). This file is mostly constants, but it does have two import-time side effects that are idempotent and rerun-safe: os.makedirs(APP_DATA_DIR, exist_ok=True) and a best-effort one-time copy from _OLD_LOG_PATH to LOG_PATH. Paths under DATA_DIR="data" are intentionally relative (relying on the app running inside the Extensions working directory via ext_cwd()), which keeps the Extensions layout portable but can misbehave if imported from a different CWD. (2) ML debuggability (log intermediate signals to localize failures: retrieval vs ranking vs synthesis). It doesn't log pipeline internals, but it centralizes the knobs that most affect retrieval behavior and reproducibility (EMBED_DIM, DEFAULT_K, NPROBE, PRECISION_THRESHOLD, phrase/bigram bonuses) and makes an explicit runtime decision to force DEVICE="cpu" to avoid CUDA/PyTorch compatibility issues—reducing "it works on my machine" ambiguity. (3) Product alignment (prioritize citations, timestamps, and 'why this result' cues to build user trust). It ensures the user-facing analytics log is stored in a stable, app-level location (PROJECT_ROOT/data/...) rather than inside the extension folder, and it preserves the timestamp-centric pipeline by defining canonical paths for transcripts, segments metadata, and course materials that downstream UIs use for "where in the lecture" navigation.

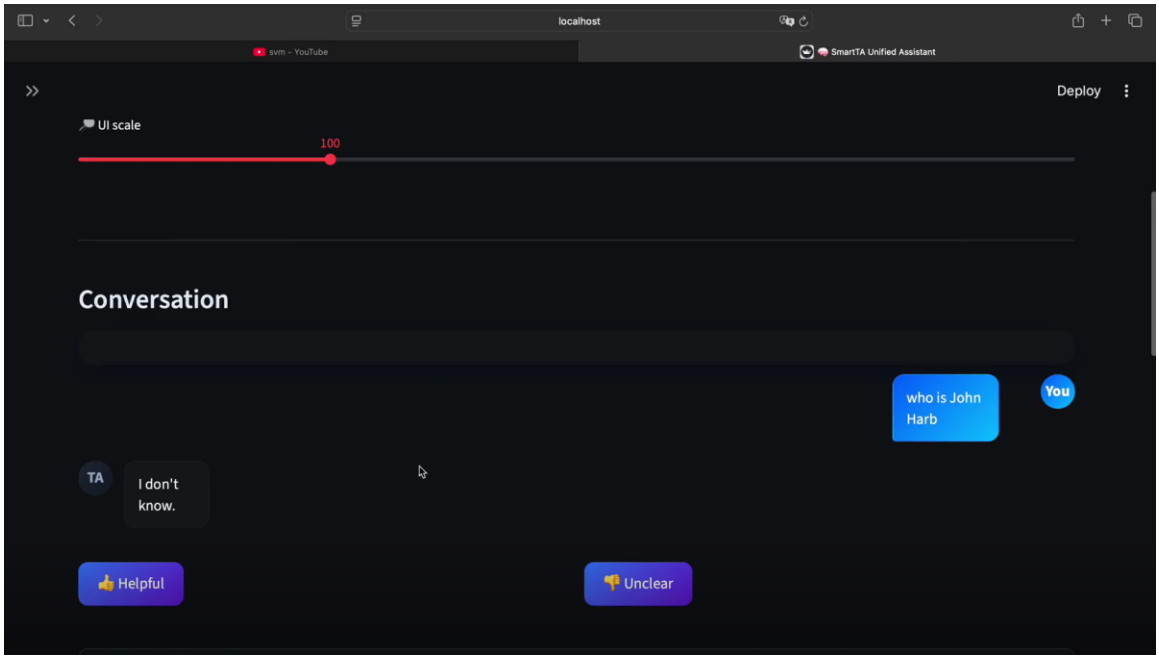
Key dependencies:
• shutil (used for migrating the legacy queries log into the unified LOG_PATH)
• torch (imported for runtime compatibility context; device is currently forced to CPU)

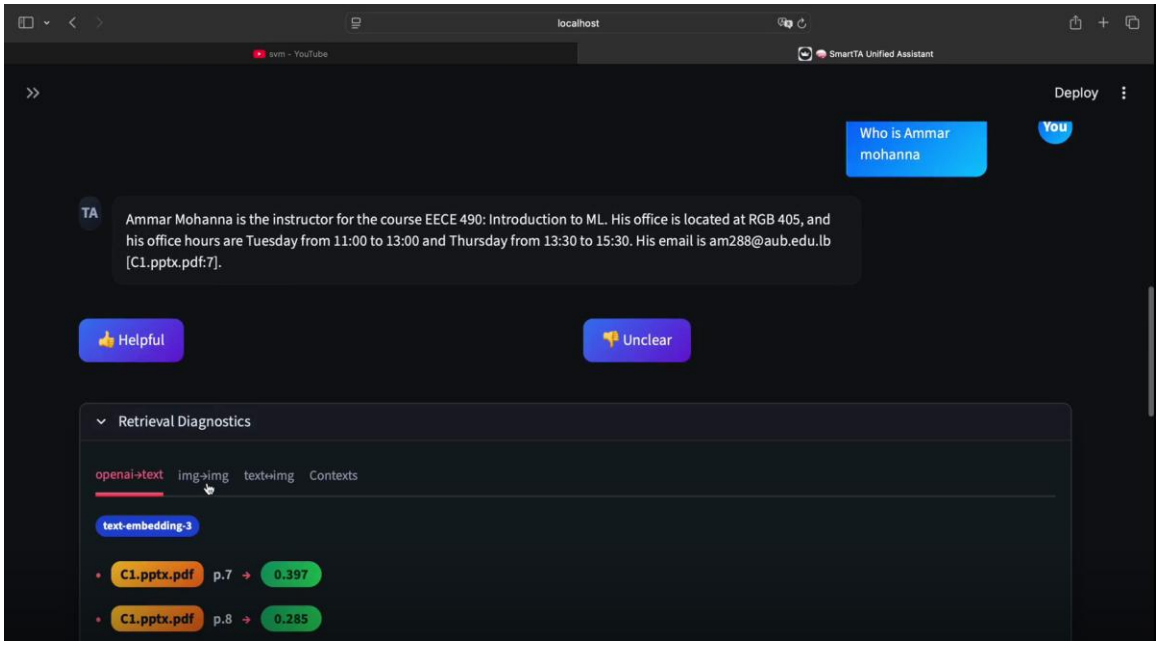
Outcomes:
• validate that APP_DATA_DIR is writable, that LOG_PATH's parent exists, and that EMBED_DIM matches the actual sentence-transformer embedding dimension used by the index (or expose a single source-of-truth dimension from the embedding model at index-build time).
• define a versioned event schema for the YouTube queries log (required keys + types for timestamp/question/lecture_filter/results) and have backend.analytics.log_query() validate against it before writing.
• keep this module "constants only," but add a small "config inspection" dict that upstream cache keys can include (so cached artifacts invalidate when EMBED_DIM/NPROBE/thresholds change). Also remove unused imports to keep import overhead minimal during reruns.
• provide a config_healthcheck() (or a centralized healthcheck elsewhere) that verifies key directories exist, legacy log migration status, FFMPEG_PATH_DEFAULT resolvability on PATH, and that index/metadata files (if present) are mutually consistent—returning PASS/FAIL + the exact remediation step.

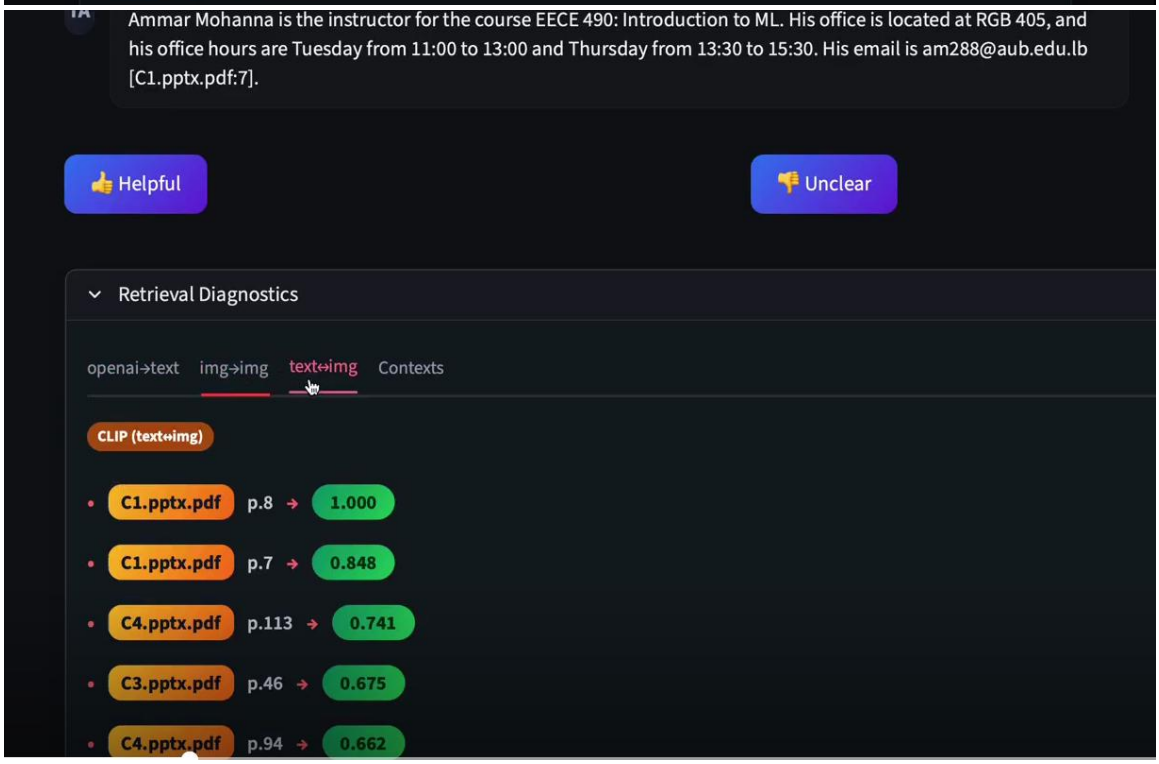
6. Results

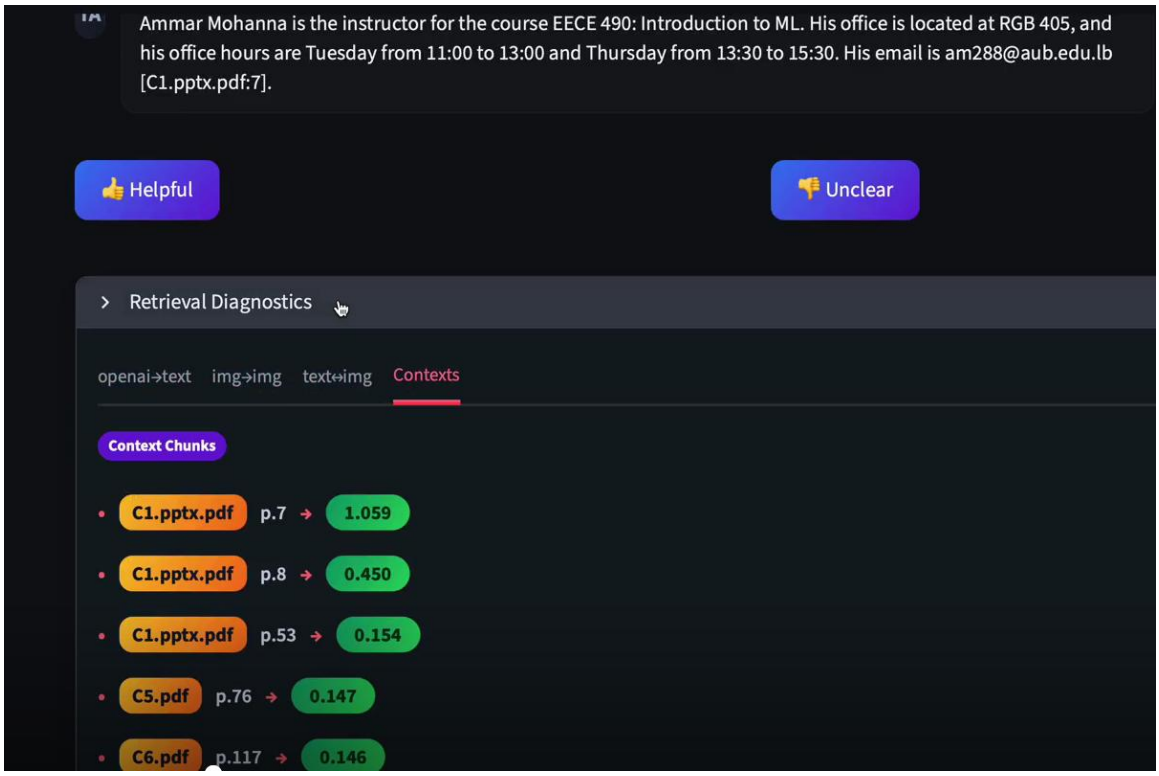
Quickstart in README with one docker run command + screenshots of the UI.

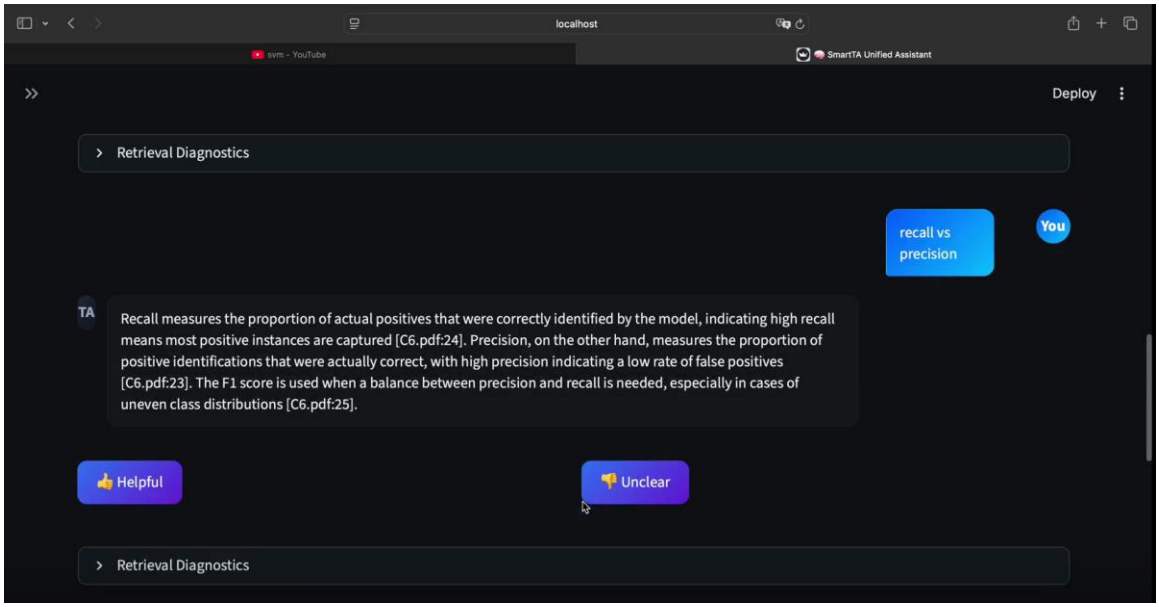


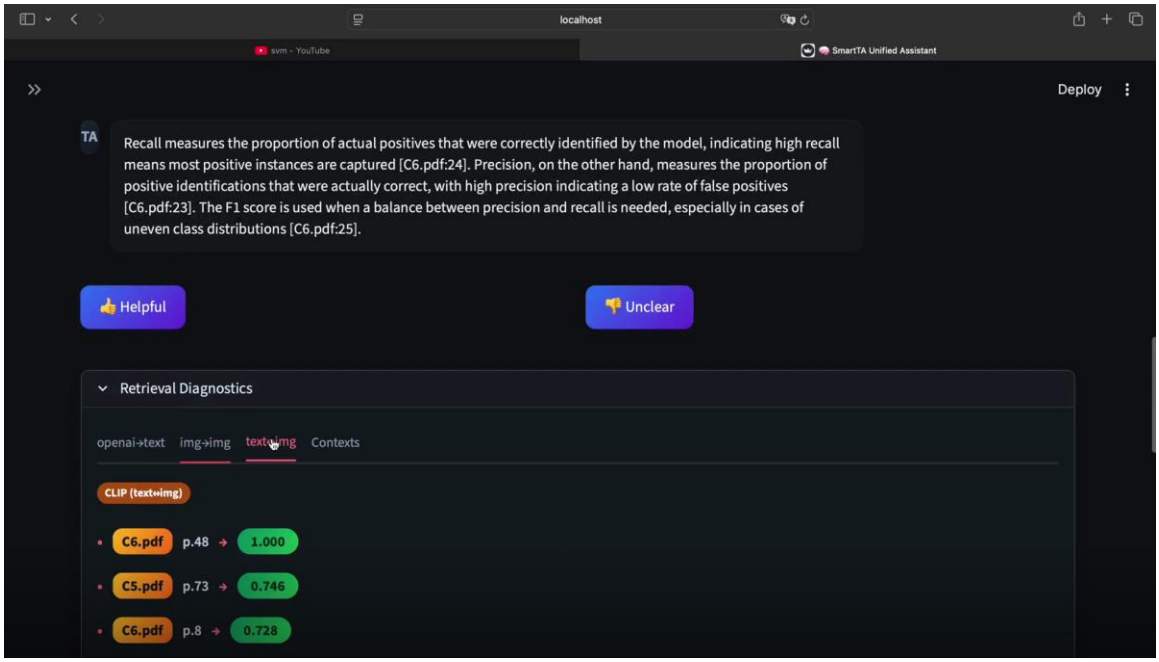


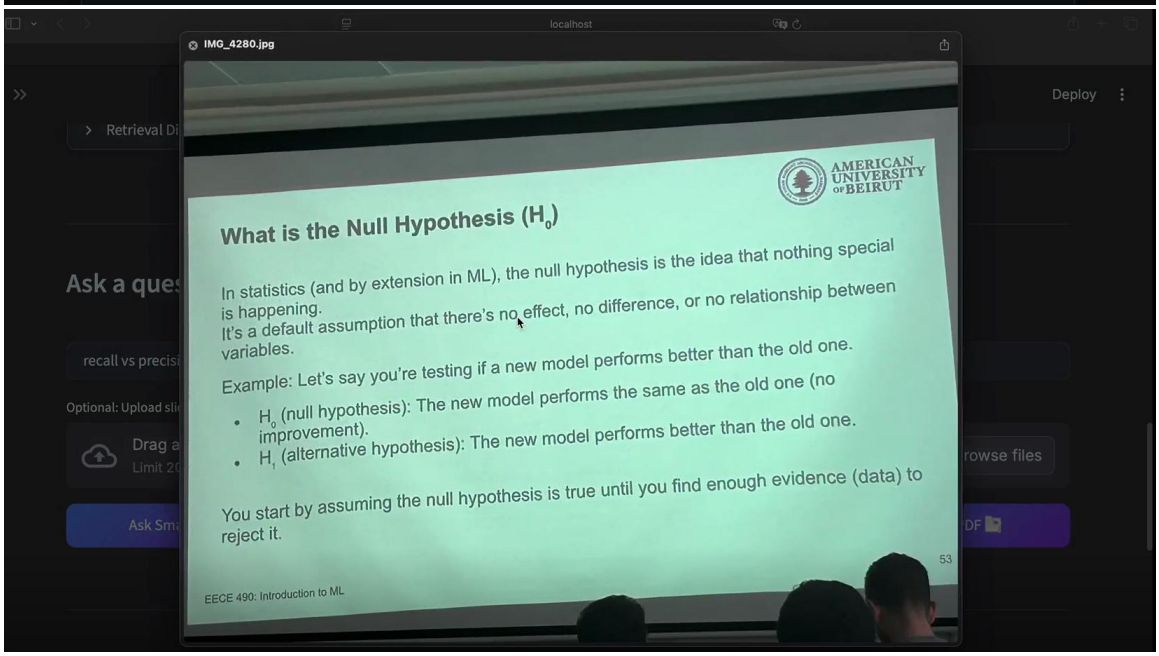

localhost
svm - YouTube
SmartTA Unified Assistant
Deploy
You
Who is Ammar mohanna
TA
Ammar Mohanna is the instructor for the course EECE 490: Introduction to ML. His office is located at RGB 405, and his office hours are Tuesday from 11:00 to 13:00 and Thursday from 13:30 to 15:30. His email is am288@aub.edu.lb [C1.pptx.pdf:7].
Helpful
Unclear
Retrieval Diagnostics
openai→text
img→img
text↔img
Contexts
text-embedding-3
C1.pptx.pdf p.7 0.397
C1.pptx.pdf p.8 0.285

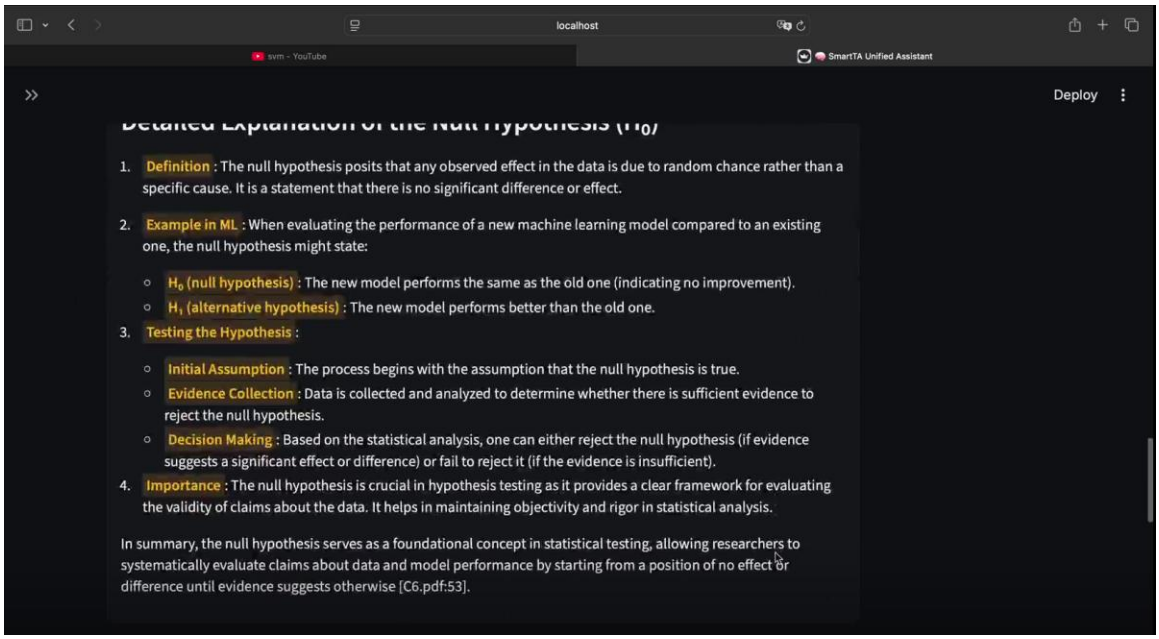

TA
Ammar Mohanna is the instructor for the course EECE 490: Introduction to ML. His office is located at RGB 405, and his office hours are Tuesday from 11:00 to 13:00 and Thursday from 13:30 to 15:30. His email is am288@aub.edu.lb [C1.pptx.pdf:7].
Helpful
Unclear
Retrieval Diagnostics
openai→text
img→img
text↔img
Contexts
CLIP (text↔img)
C1.pptx.pdf p.8 1.000
C1.pptx.pdf p.7 0.848
C4.pptx.pdf p.113 0.741
C3.pptx.pdf p.46 0.675
C4.pptx.pdf p.94 0.662

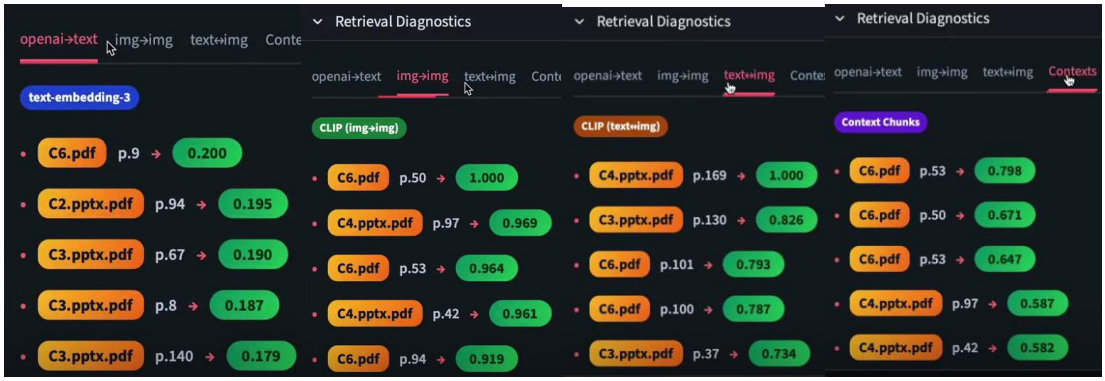

Ammar Mohanna is the instructor for the course EECE 490: Introduction to ML. His office is located at RGB 405, and his office hours are Tuesday from 11:00 to 13:00 and Thursday from 13:30 to 15:30. His email is am288@aub.edu.lb [C1.pptx.pdf:7].
Helpful
Unclear
Retrieval Diagnostics
openai→text
img→img
text↔img
Contexts
Context Chunks
C1.pptx.pdf p.7 1.059
C1.pptx.pdf p.8 0.450
C1.pptx.pdf p.53 0.154
C5.pdf p.76 0.147
C6.pdf p.117 0.146

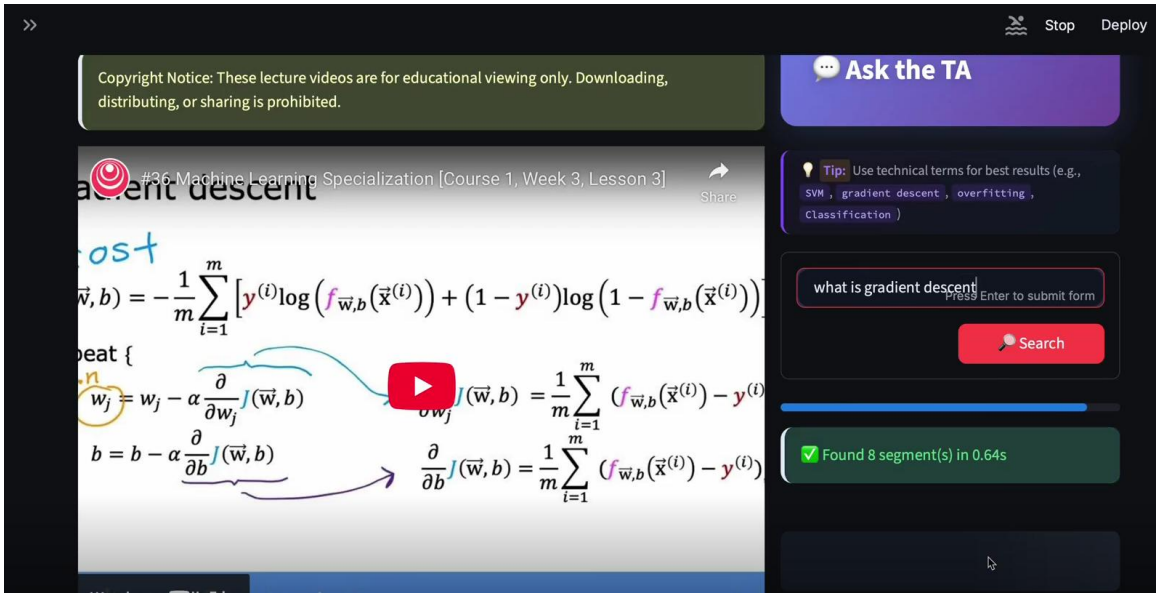

localhost
svm - YouTube
SmartTA Unified Assistant
Deploy
Retrieval Diagnostics
recall vs precision
You
TA
Recall measures the proportion of actual positives that were correctly identified by the model, indicating high recall means most positive instances are captured [C6.pdf:24]. Precision, on the other hand, measures the proportion of positive identifications that were actually correct, with high precision indicating a low rate of false positives [C6.pdf:23]. The F1 score is used when a balance between precision and recall is needed, especially in cases of uneven class distributions [C6.pdf:25].
Helpful
Unclear
Retrieval Diagnostics

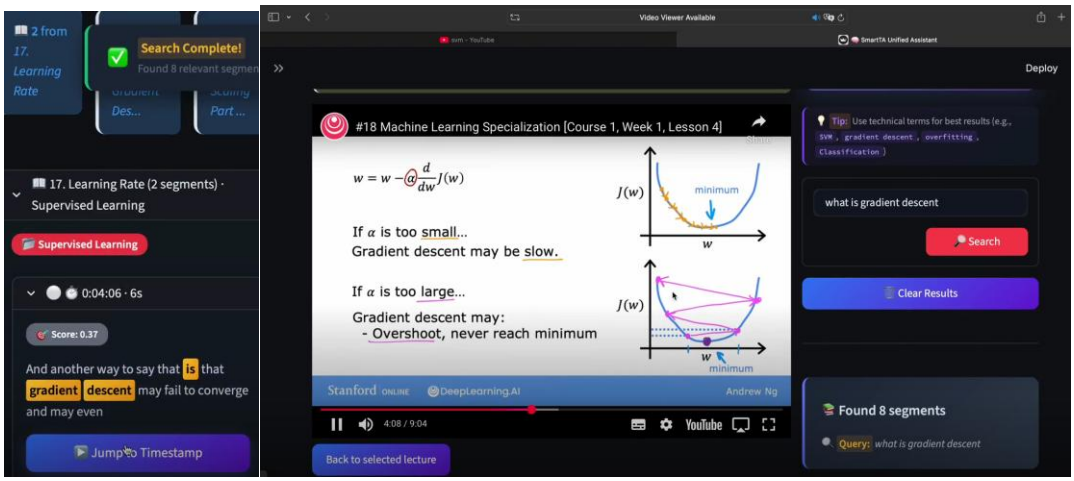

localhost
svm - YouTube
SmartTA Unified Assistant
Deploy
TA
Recall measures the proportion of actual positives that were correctly identified by the model, indicating high recall means most positive instances are captured [C6.pdf:24]. Precision, on the other hand, measures the proportion of positive identifications that were actually correct, with high precision indicating a low rate of false positives [C6.pdf:23]. The F1 score is used when a balance between precision and recall is needed, especially in cases of uneven class distributions [C6.pdf:25].
Helpful
Unclear
Retrieval Diagnostics
openai→text
img→img
text→img
Contexts
CLIP (text→img)
C6.pdf p.48 1.000
C5.pdf p.73 0.746
C6.pdf p.8 0.728

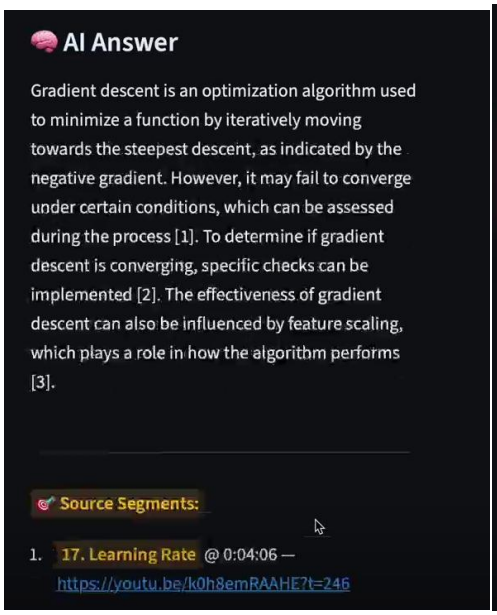

localhost
IMG_4280.jpg
Deploy
Ask a ques
recall vs precisi
Optional: Upload sli
Drag a
Limit 20
rowse files
Ask Sma
DF
AMERICAN UNIVERSITY OF BEIRUT
What is the Null Hypothesis (H_0)
In statistics (and by extension in ML), the null hypothesis is the idea that nothing special is happening.
It's a default assumption that there's no effect, no difference, or no relationship between variables.
Example: Let's say you're testing if a new model performs better than the old one.
H_0 (null hypothesis): The new model performs the same as the old one (no improvement).
H_1 (alternative hypothesis): The new model performs better than the old one.
You start by assuming the null hypothesis is true until you find enough evidence (data) to reject it.
53
EECE 490: Introduction to ML

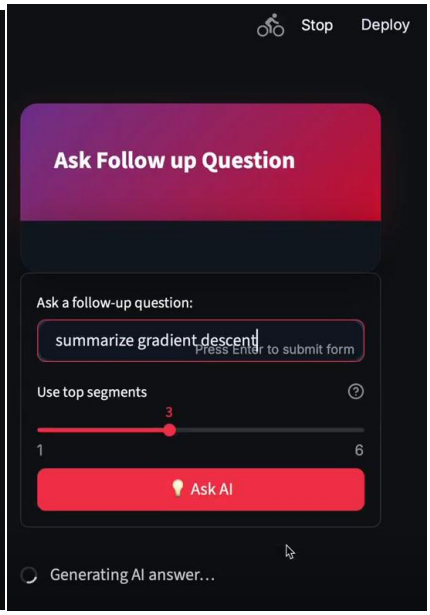

localhost
svm - YouTube
SmartTA Unified Assistant
Deploy
Detailed Explanation of the Null Hypothesis (H_0)
1. Definition : The null hypothesis posits that any observed effect in the data is due to random chance rather than a specific cause. It is a statement that there is no significant difference or effect.
2. Example in ML : When evaluating the performance of a new machine learning model compared to an existing one, the null hypothesis might state:
   - H_0 (null hypothesis) : The new model performs the same as the old one (indicating no improvement).
   - H_1 (alternative hypothesis) : The new model performs better than the old one.
3. Testing the Hypothesis :
   - Initial Assumption : The process begins with the assumption that the null hypothesis is true.
   - Evidence Collection : Data is collected and analyzed to determine whether there is sufficient evidence to reject the null hypothesis.
   - Decision Making : Based on the statistical analysis, one can either reject the null hypothesis (if evidence suggests a significant effect or difference) or fail to reject it (if the evidence is insufficient).
4. Importance : The null hypothesis is crucial in hypothesis testing as it provides a clear framework for evaluating the validity of claims about the data. It helps in maintaining objectivity and rigor in statistical analysis.
In summary, the null hypothesis serves as a foundational concept in statistical testing, allowing researchers to systematically evaluate claims about data and model performance by starting from a position of no effect or difference until evidence suggests otherwise [C6.pdf:53].

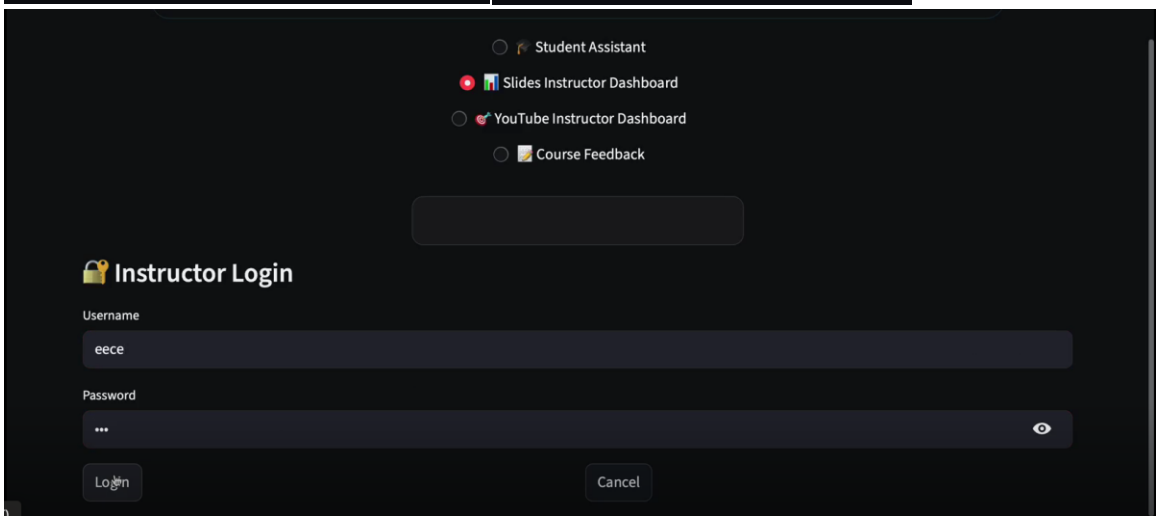

openai→text
img→img
text↔img
Conte
text-embedding-3
C6.pdf p.9 0.200
C2.pptx.pdf p.94 0.195
C3.pptx.pdf p.67 0.190
C3.pptx.pdf p.8 0.187
C3.pptx.pdf p.140 0.179
Retrieval Diagnostics
CLIP (img→img)
C6.pdf p.50 1.000
C4.pptx.pdf p.97 0.969
C6.pdf p.53 0.964
C4.pptx.pdf p.42 0.961
C6.pdf p.94 0.919
Retrieval Diagnostics
CLIP (text↔img)
C4.pptx.pdf p.169 1.000
C3.pptx.pdf p.130 0.826
C6.pdf p.101 0.793
C6.pdf p.100 0.787
C3.pptx.pdf p.37 0.734
Retrieval Diagnostics
Contexts
Context Chunks
C6.pdf p.53 0.798
C6.pdf p.50 0.671
C6.pdf p.53 0.647
C4.pptx.pdf p.97 0.587
C4.pptx.pdf p.42 0.582

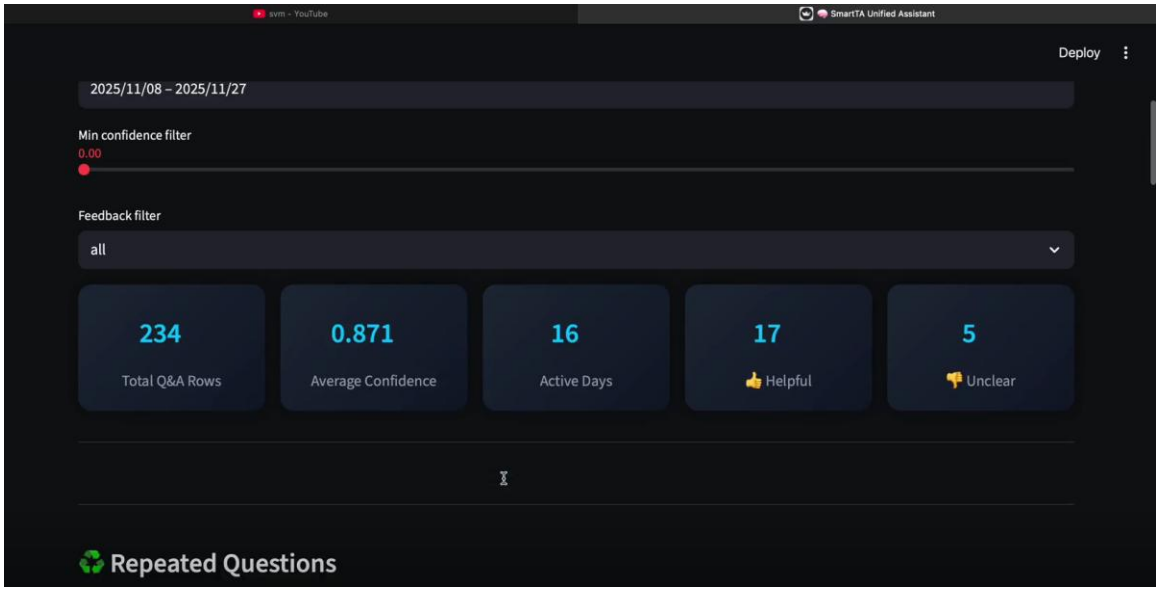

Stop
Deploy
Copyright Notice: These lecture videos are for educational viewing only. Downloading, distributing, or sharing is prohibited.
#36 Machine Learning Specialization [Course 1, Week 3, Lesson 3]
Share
Ask the TA
Tip: Use technical terms for best results (e.g., SVM , gradient descent , overfitting , Classification)
what is gradient descent
Press Enter to submit form
Search
Found 8 segment(s) in 0.64s

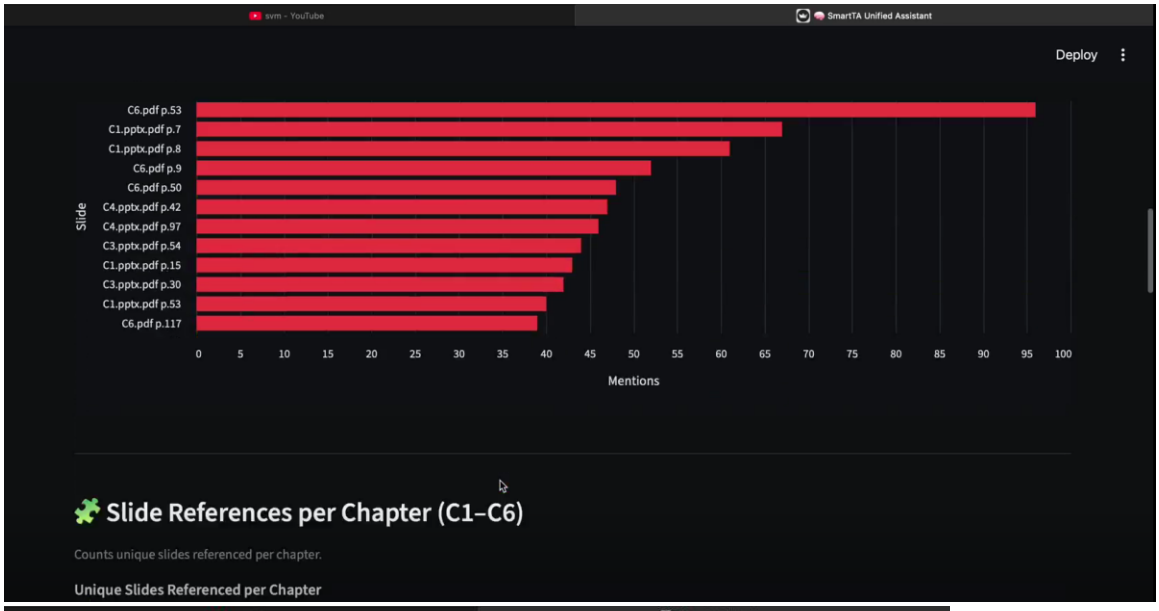

2 from 17. Learning Rate
Search Complete!
Found 8 relevant segmen
17. Learning Rate (2 segments) · Supervised Learning
Supervised Learning
0:04:06 · 6s
Score: 0.37
And another way to say that is that gradient descent may fail to converge and may even
Jump to Timestamp
Video Viewer Available
SmartTA Unified Assistant
Deploy
#18 Machine Learning Specialization [Course 1, Week 1, Lesson 4]
$w = w - \alpha \frac{d}{dw} J(w)$
If α is too small...
Gradient descent may be slow.
If α is too large...
Gradient descent may:
- Overshoot, never reach minimum
$J(w)$
minimum
w
Stanford ONLINE
DeepLearning.AI
Andrew Ng
4:08 / 9:04
YouTube
Back to selected lecture
Tip: Use technical terms for best results (e.g., SVM , gradient descent , overfitting , Classification)
what is gradient descent
Search
Clear Results
Found 8 segments
Query: what is gradient descent

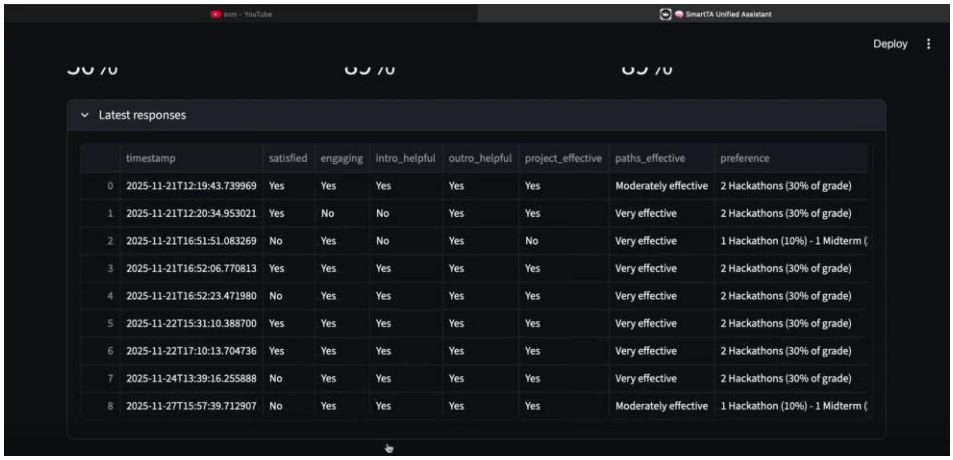

AI Answer
Gradient descent is an optimization algorithm used to minimize a function by iteratively moving towards the steepest descent, as indicated by the negative gradient. However, it may fail to converge under certain conditions, which can be assessed during the process [1]. To determine if gradient descent is converging, specific checks can be implemented [2]. The effectiveness of gradient descent can also be influenced by feature scaling, which plays a role in how the algorithm performs [3].
Source Segments:
1. 17. Learning Rate @ 0:04:06 — https://youtu.be/k0h8emRAAHE?t=246


Stop
Deploy
Ask Follow up Question
Ask a follow-up question:
summarize gradient descent
Press Enter to submit form
Use top segments
3
1
6
Ask AI
Generating AI answer…


Student Assistant
Slides Instructor Dashboard
YouTube Instructor Dashboard
Course Feedback
Instructor Login
Username
eece
Password
Login
Cancel





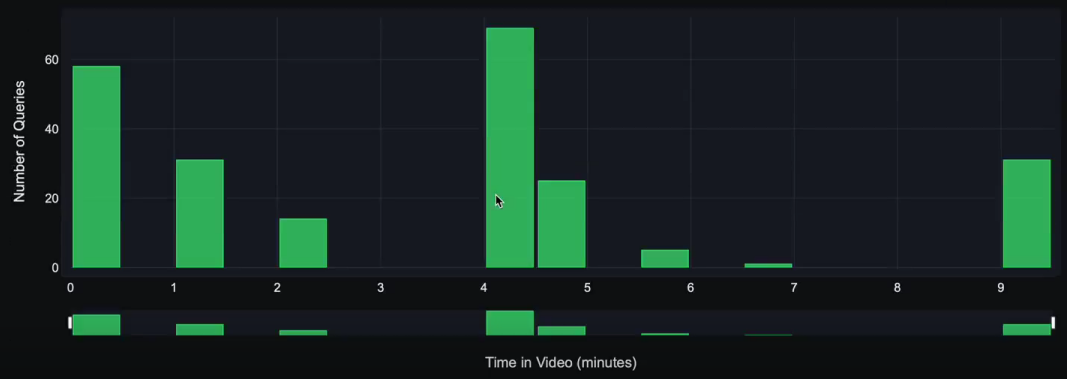


	timestamp	satisfied	engaging	intro_helpful	outro_helpful	project_effective	paths_effective	preference
0	2025-11-21T12:19:43.739969	Yes	Yes	Yes	Yes	Yes	Moderately effective	2 Hackathons (30% of grade)
1	2025-11-21T12:20:34.953021	Yes	No	No	Yes	Yes	Very effective	2 Hackathons (30% of grade)
2	2025-11-21T16:51:51.083269	No	Yes	No	Yes	No	Very effective	1 Hackathon (10%) - 1 Midterm (
3	2025-11-21T16:52:06.770813	Yes	Yes	Yes	Yes	Yes	Very effective	2 Hackathons (30% of grade)
4	2025-11-21T16:52:23.471980	No	Yes	Yes	Yes	Yes	Very effective	2 Hackathons (30% of grade)
5	2025-11-22T15:31:10.388700	Yes	Yes	Yes	Yes	Yes	Very effective	2 Hackathons (30% of grade)
6	2025-11-22T17:10:13.704736	Yes	Yes	Yes	Yes	Yes	Very effective	2 Hackathons (30% of grade)
7	2025-11-24T13:39:16.255888	No	Yes	Yes	Yes	Yes	Very effective	2 Hackathons (30% of grade)
8	2025-11-27T15:57:39.712907	No	Yes	Yes	Yes	Yes	Moderately effective	1 Hackathon (10%) - 1 Midterm (

Query Timeline Analysis

Graph 1 of 35

Query Distribution: 17. Learning Rate



Course Feedback Snapshot

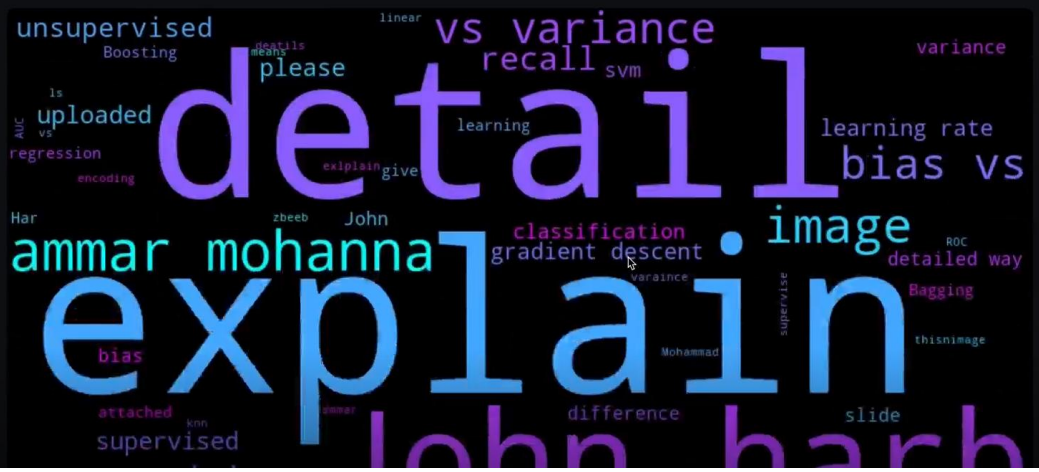
Collected responses: 9

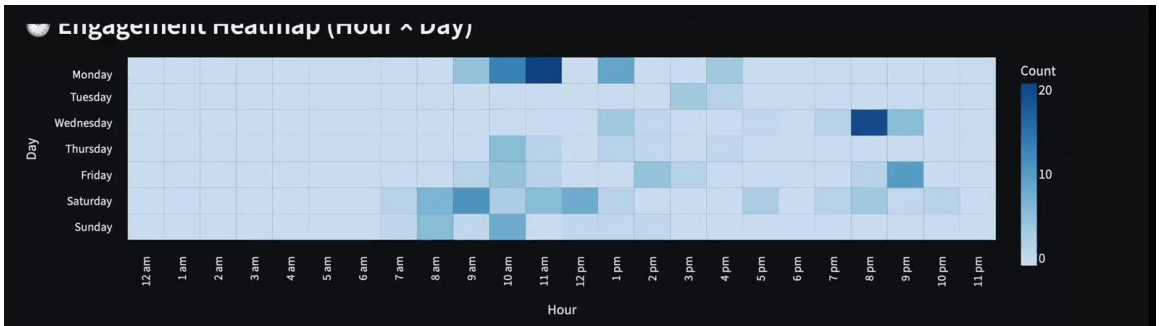
Satisfied	Engaged in class	Project approach effective
56%	89%	89%

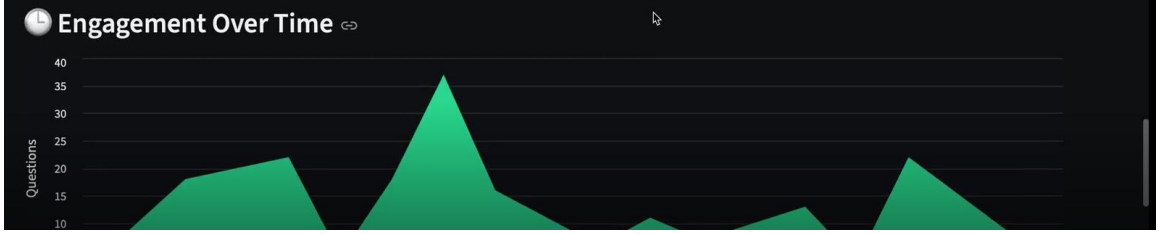
› Latest responses

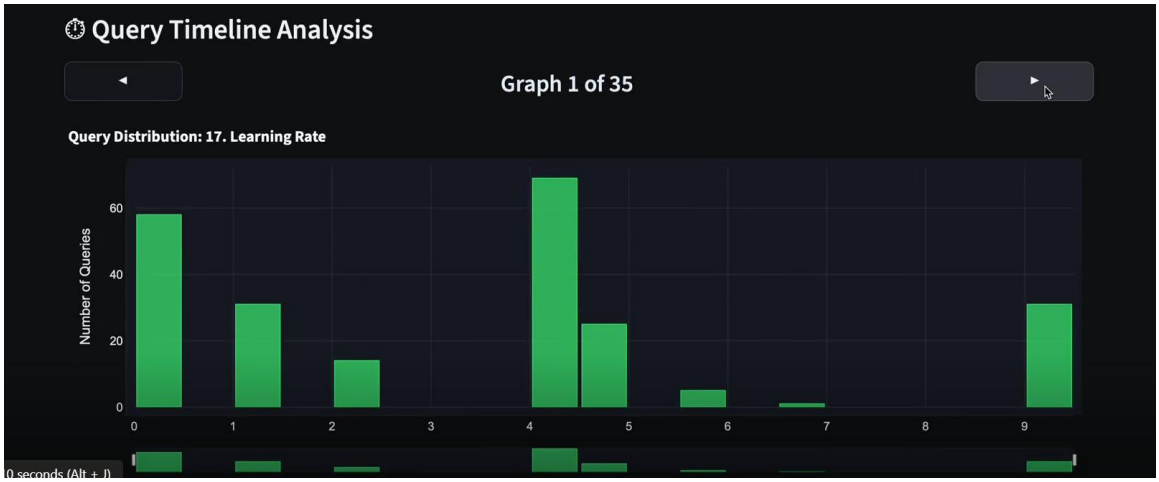
Common Words in Student Questions

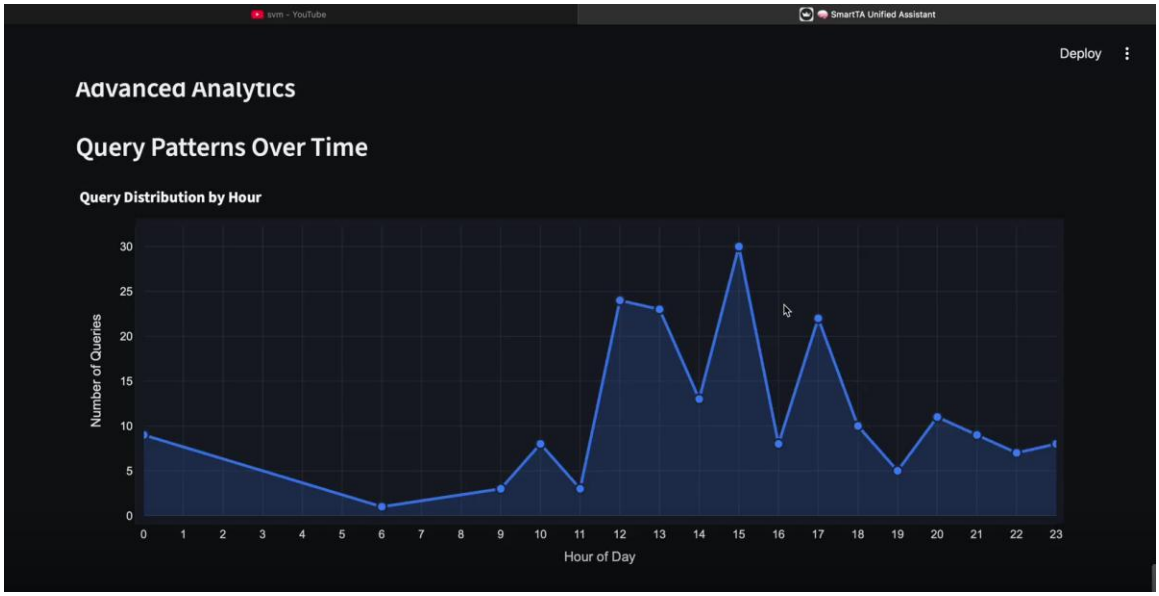
Shows most frequent keywords extracted from student questions.

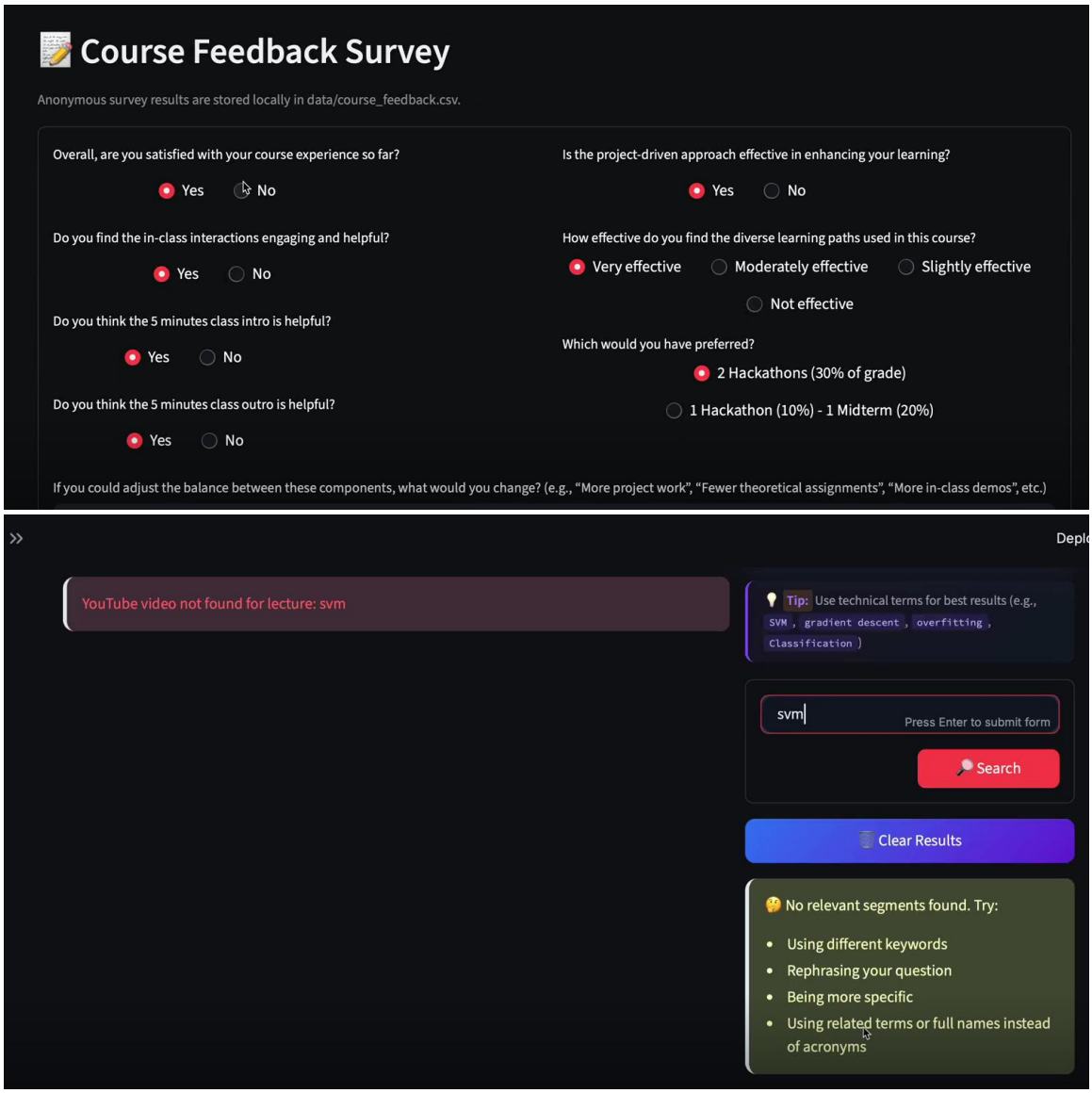



Engagement Heatmap (Hour x Day)
Monday
Tuesday
Wednesday
Thursday
Friday
Saturday
Sunday
Day
Count
20
10
0
12 am
1 am
2 am
3 am
4 am
5 am
6 am
7 am
8 am
9 am
10 am
11 am
12 pm
1 pm
2 pm
3 pm
4 pm
5 pm
6 pm
7 pm
8 pm
9 pm
10 pm
11 pm
Hour


Engagement Over Time
40
35
30
25
20
15
10
Questions


Query Timeline Analysis
Graph 1 of 35
Query Distribution: 17. Learning Rate
Number of Queries
60
40
20
0
0
1
2
3
4
5
6
7
8
9


svm - YouTube
SmartTA Unified Assistant
Deploy
Advanced Analytics
Query Patterns Over Time
Query Distribution by Hour
Number of Queries
30
25
20
15
10
5
0
0
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
Hour of Day



Below are figures showing the multi-channel retrieval logic when prompting the product to explain the concept of gradient descent, for instance, based on the course lecture deliverable:

Deploy

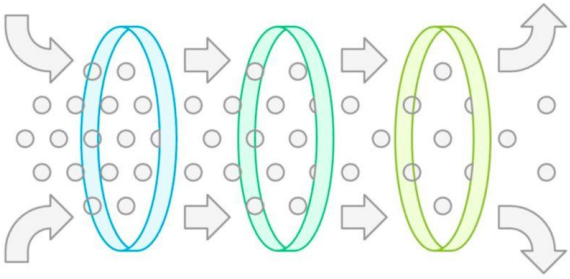
explain in details

You

Optimization Algorithms - Gradient Descent

AMERICAN UNIVERSITY OF BEIRUT

Thus, the update moves as follows:



Calculate Derivative	Adjust Parameters	Iterate Process
Assess slope of loss	Move closer to	Repeat until

Deploy

Calculate Derivative	Adjust Parameters	Iterate Process
Assess slope of loss function	Move closer to minimum	Repeat until convergence

Attached image

TA

Gradient Descent (GD) is a fundamental optimization algorithm used in machine learning to minimize the loss function, which quantifies the error of a model's predictions. The process is iterative and involves several key steps:

1. **Initialization**: The algorithm starts at an initial point, denoted as x_0, which represents the initial values of the model parameters.
2. **Calculate Derivative**: At each iteration, the algorithm calculates the derivative of the loss function with respect to the model parameters. This derivative indicates the slope of the loss function at the current point, helping to determine the direction in which to adjust the parameters to reduce the loss [C4.pptx.pdf:60].
3. **Adjust Parameters**: Based on the calculated derivative, the parameters are updated. The update rule typically follows the formula: $x_{new} = x_{old} - \alpha \cdot \nabla L(x_{old})$ where α is the learning rate, and $\nabla L(x_{old})$ is the gradient of the loss function at the current parameters. The learning rate controls the size of the steps taken towards the minimum [C4.pptx.pdf:61].
4. **Iterate Process**: The process of calculating the derivative and adjusting the parameters is repeated iteratively until convergence is achieved, meaning that the changes in the loss function are minimal or the parameters stabilize [C4.pptx.pdf:60].
5. **Types of Gradient Descent**: The updates can be performed in different ways:
   - **Stochastic Gradient Descent (SGD)**: Updates are made after each individual data point.
   - **Batch Gradient Descent**: Updates occur after processing a batch of data points.
   - **Gradient Descent**: Updates are made after processing the entire dataset [C4.pptx.pdf:66].

The choice of the learning rate is crucial, as it affects the convergence speed and stability of the algorithm. A learning rate that is too high may cause the algorithm to overshoot the minimum, while a rate that is too low may result in slow convergence [C4.pptx.pdf:63].

In summary, Gradient Descent is an essential algorithm for optimizing machine learning models by iteratively adjusting parameters to minimize the loss function.

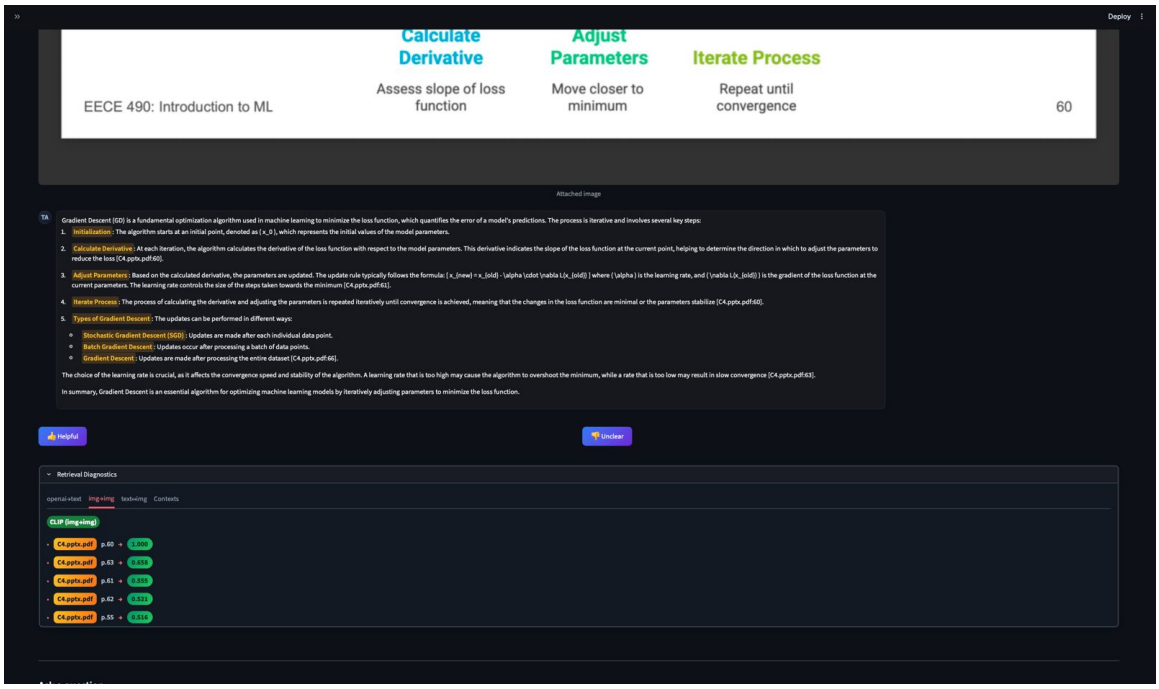
👍 Helpful 👎 Unclear

Retrieval Diagnostics

openai-vtext img-vimg textvimg Contexts

Context Chunks

- C4.pptx.pdf p.60 → 0.795
- C4.pptx.pdf p.63 → 0.615
- C4.pptx.pdf p.61 → 0.404
- C4.pptx.pdf p.66 → 0.390
- C4.pptx.pdf p.58 → 0.386



Appendix A — Repository Manifest (Files, Sizes, Hashes)

Every file in the Git repo is listed below for auditability.

Path	Size (KB)	Ext	Hash
.gitignore	0.4	(no ext)	d66c914939
README.md	11.3	.md	bd1a7cc38c
SmartTA_Extensions/backend/__init__.py	0.8	.py	8b84645fbd
SmartTA_Extensions/backend/analytics.py	18.2	.py	83cc5c550c
SmartTA_Extensions/backend/indexing.py	6.0	.py	16f5c77e33
SmartTA_Extensions/backend/resources.py	5.1	.py	c4daa6f67f
SmartTA_Extensions/backend/search.py	28.1	.py	ad15998acb
SmartTA_Extensions/backend/transcription.py	10.9	.py	5688d549cc
SmartTA_Extensions/backend/video.py	3.4	.py	3e9de57389
SmartTA_Extensions/data/course.index	4,366.5	.index	9e70d63e6f
SmartTA_Extensions/data/course_materials.json	0.0	.json	97d170e155
SmartTA_Extensions/data/gold_queries.json	0.5	.json	aef019d999
SmartTA_Extensions/data/lectures/01_what_is_machine_learning.json	0.3	.json	b52cbd9c88
SmartTA_Extensions/data/lectures/02_applications_of_machine_learning.json	0.3	.json	c4573361cc
SmartTA_Extensions/data/lectures/03_supervised_learning_part_1.json	0.3	.json	be3b45bfaa

SmartTA_Extensions/data/lectures/04_supervised_learning_part_2.json	0.3	.json	1338f29e83
SmartTA_Extensions/data/lectures/05_unsupervised_learning_part_1.json	0.3	.json	f24f5a4a71
SmartTA_Extensions/data/lectures/06_unsupervised_learning_part_2.json	0.3	.json	6dccc7d501
SmartTA_Extensions/data/lectures/07_jupyter_notebooks.json	0.2	.json	f319987e0b
SmartTA_Extensions/data/lectures/08_linear_regression_model_part_1.json	0.3	.json	6b9d2a11e3
SmartTA_Extensions/data/lectures/09_linear_regression_model_part_2.json	0.3	.json	2057106ca7
SmartTA_Extensions/data/lectures/10_cost_function.json	0.2	.json	728ed28d8b
SmartTA_Extensions/data/lectures/11_cost_function_intuition.json	0.3	.json	685ed232ce
SmartTA_Extensions/data/lectures/12_visualizing_the_cost_function.json	0.3	.json	28233b8511
SmartTA_Extensions/data/lectures/13_visualization_examples.json	0.3	.json	33c66b98aa
SmartTA_Extensions/data/lectures/14_gradient_descent.json	0.2	.json	9c73132264
SmartTA_Extensions/data/lectures/15_implementing_gradient_descent.json	0.3	.json	350707cb3a
SmartTA_Extensions/data/lectures/16_gradient_descent_intuition.json	0.3	.json	e0b31de7db
SmartTA_Extensions/data/lectures/17_learning_rate.json	0.2	.json	d0bccaaf29
SmartTA_Extensions/data/lectures/18_gradient_descent_for_linear_regression.json	0.3	.json	9b783182e8
SmartTA_Extensions/data/lectures/19_multiple_features.json	0.3	.json	5bdb56ba75
SmartTA_Extensions/data/lectures/20_vectorization_part_1.json	0.3	.json	340ddcd1fe
SmartTA_Extensions/data/lectures/21_vectorization_part_2.json	0.3	.json	b87876a83b
SmartTA_Extensions/data/lectures/22_gradient_descent_for_multiple_regression.json	0.3	.json	326aa9fe6a
SmartTA_Extensions/data/lectures/23_feature_scaling_part_1.json	0.3	.json	50d8e6044d
SmartTA_Extensions/data/lectures/24_feature_scaling_part_2.json	0.3	.json	50e2fe37f8
SmartTA_Extensions/data/lectures/25_checking_gradient_descent_for_convergence.json	0.3	.json	74e1dd8939
SmartTA_Extensions/data/lectures/26_choosing_the_learning_rate.json	0.3	.json	1ac6e14dc0
SmartTA_Extensions/data/lectures/27_feature_engineering.json	0.3	.json	fdb18486a5
SmartTA_Extensions/data/lectures/28_polynomial_regression.json	0.3	.json	c050ed30c2

SmartTA_Extensions/data/lectures/29_motivation.json	0.2	.json	e40bb62d4b
SmartTA_Extensions/data/lectures/30_logistic_regression.json	0.3	.json	c348256719
SmartTA_Extensions/data/lectures/31_decision_boundary.json	0.2	.json	7d6d950622
SmartTA_Extensions/data/lectures/32_cost_function_for_logistic_regression.json	0.3	.json	48d3ebbd77
SmartTA_Extensions/data/lectures/33_simplified_cost_function.json	0.3	.json	11d06d1e05
SmartTA_Extensions/data/lectures/34_gradient_descent_implementation.json	0.3	.json	97ba1cdbce
SmartTA_Extensions/data/lectures/35_the_problem_of_overfitting.json	0.3	.json	d3c5e0df35
SmartTA_Extensions/data/lectures/36_addressing_overfitting.json	0.3	.json	65bc4a6aae
SmartTA_Extensions/data/lectures/37_cost_function_with_regularization.json	0.3	.json	abc843ae11
SmartTA_Extensions/data/lectures/38_regularized_linear_regression.json	0.3	.json	8e8bc7ae4f
SmartTA_Extensions/data/lectures/39_regularized_logistic_regression.json	0.3	.json	c9657e7413
SmartTA_Extensions/data/metadata.json	575.7	.json	f03d049f6a
SmartTA_Extensions/data/queries_log.json	126.0	.json	8a795b3bea
SmartTA_Extensions/data/segments_metadata.json	461.6	.json	1d4c620b54
SmartTA_Extensions/data/transcripts/01. What is Machine Learning.txt	5.0	.txt	26902b5411
SmartTA_Extensions/data/transcripts/02. Applications of Machine Learning.txt	3.9	.txt	6df14ee664
SmartTA_Extensions/data/transcripts/03. Supervised Learning Part 1.txt	5.6	.txt	583b95f3e0
SmartTA_Extensions/data/transcripts/04. Supervised Learning Part 2.txt	5.6	.txt	d2bfc93d9f
SmartTA_Extensions/data/transcripts/05. Unsupervised Learning Part 1.txt	7.3	.txt	c47557394b
SmartTA_Extensions/data/transcripts/06. Unsupervised Learning Part 2.txt	3.1	.txt	d50cc2477c
Path	Size (KB)	Ext	Hash
SmartTA_Extensions/data/transcripts/07. Jupyter Notebooks.txt	3.9	.txt	86342ae98f
SmartTA_Extensions/data/transcripts/08. Linear Regression Model Part 1.txt	7.6	.txt	dae00836ad
SmartTA_Extensions/data/transcripts/09. Linear Regression Model Part 2.txt	5.0	.txt	b02144f8dd
SmartTA_Extensions/data/transcripts/10. Cost Function.txt	6.3	.txt	a64f3e17da

SmartTA_Extensions/data/transcripts/11. Cost Function Intuition.txt	10.3	.txt	6efcd5f202
SmartTA_Extensions/data/transcripts/12. Visualizing The Cost Function.txt	6.3	.txt	01416ce8cf
SmartTA_Extensions/data/transcripts/13. Visualization Examples.txt	4.6	.txt	6bd029e269
SmartTA_Extensions/data/transcripts/14. Gradient Descent.txt	6.4	.txt	2f9715cca0
SmartTA_Extensions/data/transcripts/15. Implementing Gradient Descent.txt	7.5	.txt	9867128e03
SmartTA_Extensions/data/transcripts/16. Gradient Descent Intuition.txt	5.4	.txt	5385a2c125
SmartTA_Extensions/data/transcripts/17. Learning Rate.txt	6.9	.txt	bd4d96a4a4
SmartTA_Extensions/data/transcripts/18. Gradient Descent For Linear Regression.txt	5.0	.txt	0a05ddcedb
SmartTA_Extensions/data/transcripts/19. Multiple Features.txt	6.8	.txt	24bfab25e7
SmartTA_Extensions/data/transcripts/20. Vectorization Part 1.txt	5.2	.txt	e8eb2a4b8a
SmartTA_Extensions/data/transcripts/21. Vectorization Part 2.txt	5.3	.txt	8e2b8dafdb
SmartTA_Extensions/data/transcripts/22. Gradient Descent For Multiple Regression.txt	5.9	.txt	db23ceba7b
SmartTA_Extensions/data/transcripts/23. Feature Scaling Part 1.txt	4.8	.txt	35fd341693
SmartTA_Extensions/data/transcripts/24. Feature Scaling Part 2.txt	5.5	.txt	384f843267
SmartTA_Extensions/data/transcripts/25. Checking Gradient Descent for Convergence.txt	4.4	.txt	a51c9050b1
SmartTA_Extensions/data/transcripts/26. Choosing the Learning Rate.txt	5.1	.txt	b23a20e012
SmartTA_Extensions/data/transcripts/27. Feature Engineering.txt	2.5	.txt	0848dacefd
SmartTA_Extensions/data/transcripts/28. Polynormial Regression.txt	5.0	.txt	7b5ba9e8b3
SmartTA_Extensions/data/transcripts/29. Motivation.txt	7.8	.txt	b2101fc5ba
SmartTA_Extensions/data/transcripts/30. Logistic Regression.txt	7.0	.txt	dd0d00be55
SmartTA_Extensions/data/transcripts/31. Decision Boundary.txt	7.4	.txt	3cae8bf598
SmartTA_Extensions/data/transcripts/32. Cost Function for Logistic Regression.txt	9.0	.txt	f65b60829f
SmartTA_Extensions/data/transcripts/33. Simplified Cost Function.txt	4.0	.txt	2b754f1af7
SmartTA_Extensions/data/transcripts/34. Gradient Descent Implementation.txt	5.0	.txt	1c7f315703

SmartTA_Extensions/data/transcripts/35. The Problem of Overfitting.txt	9.6	.txt	07d7f9c328
SmartTA_Extensions/data/transcripts/36. Addressing Overfitting.txt	6.8	.txt	96a7b3f7c9
SmartTA_Extensions/data/transcripts/37. Cost Function with Regularization.txt	7.2	.txt	d85bcc1fb0
SmartTA_Extensions/data/transcripts/38. Regularized Linear Regression.txt	6.3	.txt	60056130d0
SmartTA_Extensions/data/transcripts/39. Regularized Logistic Regression.txt	4.7	.txt	12cf80aa88
SmartTA_Extensions/data/uploaded_files/.gitkeep	0.0	(no ext)	da39a3ee5e
SmartTA_Extensions/frontend/__init__.py	0.0	.py	da39a3ee5e
SmartTA_Extensions/frontend/components.py	12.8	.py	5cc3524993
SmartTA_Extensions/frontend/styles.py	12.5	.py	193d559371
SmartTA_Extensions/frontend/tabs/__init__.py	0.0	.py	da39a3ee5e
SmartTA_Extensions/frontend/tabs/professor.py	40.5	.py	2f0519c947
SmartTA_Extensions/frontend/tabs/student.py	26.1	.py	f3b51e4ca0
SmartTA_Extensions/frontend/types.py	1.2	.py	f12fd2dbd0
SmartTA_Extensions/utils/__init__.py	0.0	.py	da39a3ee5e
SmartTA_Extensions/utils/check_imports.py	0.2	.py	139bb7c0a8
SmartTA_Extensions/utils/config.py	2.7	.py	2b8dafcad6
SmartTA_Extensions/utils/helpers.py	5.8	.py	765a7c8ed0
SmartTA_RAG/All_chapters_Rag.ipynb	6,215.6	.ipynb	fcaf257464
SmartTA_RAG/data/index/X_img_openai_clip_vit_large_patch14_336.npy	6,228.1	.npy	50cc3bb4cd
SmartTA_RAG/data/index/X_text.npy	8,808.1	.npy	317d709619
SmartTA_RAG/data/index/X_text_clip.npy	2,202.1	.npy	de98884078
SmartTA_RAG/data/index/chunk_meta.jsonl	35.7	.jsonl	6ac75d6a2c
SmartTA_RAG/data/index/chunks.jsonl	172.3	.jsonl	acebb42ef0
SmartTA_RAG/data/index/image_meta.jsonl	232.4	.jsonl	be209787e7
SmartTA_RAG/data/index/images_openai_clip_vit_large_patch14_336.faiss	6,244.3	.faiss	0c225d36c4
SmartTA_RAG/data/index/page_to_chunk_ids.pkl	9.7	.pkl	c1f31bd738
SmartTA_RAG/data/index/text.faiss	8,813.8	.faiss	6d85bb00dc
SmartTA_RAG/data/index/text_clip.faiss	2,207.8	.faiss	1ceb338e61
SmartTA_RAG/data/raw/C1.pptx.pdf	4,890.4	.pdf	538c7befc7
SmartTA_RAG/data/raw/C2.pptx.pdf	7,148.0	.pdf	48c3c0ac47

SmartTA_RAG/data/raw/C3.pptx.pdf	12,157.9	.pdf	ad9df8bae5
SmartTA_RAG/data/raw/C4.pptx.pdf	7,042.8	.pdf	95de17b7ef
Path	Size (KB)	Ext	Hash
SmartTA_RAG/data/raw/C5.pdf	4,468.0	.pdf	61cb581cde
SmartTA_RAG/data/raw/C6.pdf	4,014.9	.pdf	b53b35495a
SmartTA_RAG/rag_engine1.py	0.2	.py	27e803e976
data/chat_logs.ndjson	590.3	.ndjson	0a1a9860aa
data/course_feedback.csv	1.2	.csv	bc1a132014
data/youtube_queries_log.json	294.7	.json	cd2bed0f1d
dockerfile	0.4	(no ext)	7a341f257b
requirements-docker.txt	0.8	.txt	963fffc8e0
requirements.txt	4.9	.txt	280434c389
smartta_unified/__init__.py	0.1	.py	a36dbc1e01
smartta_unified/aub_logo.png	17.7	.png	e37bec33b1
smartta_unified/chat.py	18.1	.py	b71705d98a
smartta_unified/config.py	5.9	.py	0f83b0991c
smartta_unified/dashboard.py	10.9	.py	3e385a4457
smartta_unified/feedback.py	4.6	.py	171cd2d5ab
smartta_unified/helpers.py	5.4	.py	67a3c9048e
smartta_unified/main.py	5.6	.py	4fa477b92f
smartta_unified/rag/__init__.py	0.5	.py	b4c5469c86
smartta_unified/rag/embeddings.py	2.1	.py	6e1b153de4
smartta_unified/rag/engine.py	8.5	.py	2fac859658
smartta_unified/rag/indexes.py	2.5	.py	4f610395b3
smartta_unified/rag/search.py	2.8	.py	af7b026c26
smartta_unified/rag/settings.py	0.6	.py	813b415dee
smartta_unified/rag/state.py	0.5	.py	2cd6637e07
smartta_unified/style.py	6.2	.py	056cb64eeb
smartta_unified/youtube.py	3.0	.py	6633f0310d

Appendix B — Full Python Inventory (Sorted by LOC)

File	LOC	Functions	Docstring?	Hash
SmartTA_Extensions/frontend/tabs/professor.py	728	1	No	2f0519c947
SmartTA_Extensions/backend/search.py	690	11	Yes	ad15998acb

SmartTA_Extensions/frontend/tabs/student.py	515	1	No	f3b51e4ca0
smartta_unified/chat.py	510	2	No	b71705d98a
SmartTA_Extensions/backend/analytics.py	459	7	Yes	83cc5c550c
smartta_unified/dashboard.py	320	1	No	3e385a4457
SmartTA_Extensions/frontend/styles.py	272	4	Yes	193d559371
smartta_unified/rag/engine.py	254	2	Yes	2fac859658
SmartTA_Extensions/backend/transcription.py	250	4	Yes	5688d549cc
SmartTA_Extensions/frontend/components.py	244	3	No	5cc3524993
smartta_unified/config.py	207	0	Yes	0f83b0991c
SmartTA_Extensions/utils/helpers.py	185	10	Yes	765a7c8ed0
smartta_unified/helpers.py	173	6	Yes	67a3c9048e
SmartTA_Extensions/backend/indexing.py	162	4	Yes	16f5c77e33
SmartTA_Extensions/backend/resources.py	154	5	Yes	c4daa6f67f
smartta_unified/main.py	154	1	Yes	4fa477b92f
smartta_unified/style.py	110	0	Yes	056cb64eeb
smartta_unified/feedback.py	103	2	Yes	171cd2d5ab
SmartTA_Extensions/backend/video.py	99	4	Yes	3e9de57389
smartta_unified/youtube.py	88	2	Yes	6633f0310d
smartta_unified/rag/search.py	85	4	Yes	af7b026c26
SmartTA_Extensions/utils/config.py	76	0	Yes	2b8dafcad6
smartta_unified/rag/indexes.py	75	1	Yes	4f610395b3
smartta_unified/rag/embeddings.py	64	3	Yes	6e1b153de4
SmartTA_Extensions/frontend/types.py	53	0	Yes	f12fd2dbd0

SmartTA_Extensions/backend/__init__.py	39	0	Yes	8b84645fbd
smartta_unified/rag/settings.py	27	0	Yes	813b415dee
smartta_unified/rag/state.py	22	0	Yes	2cd6637e07
smartta_unified/rag/__init__.py	21	0	Yes	b4c5469c86
SmartTA_RAG/rag_engine1.py	12	0	Yes	27e803e976
SmartTA_Extensions/utils/check_imports.py	7	0	No	139bb7c0a8
smartta_unified/__init__.py	5	0	Yes	a36dbc1e01
SmartTA_Extensions/frontend/__init__.py	0	0	No	da39a3ee5e
SmartTA_Extensions/frontend/tabs/__init__.py	0	0	No	da39a3ee5e
SmartTA_Extensions/utils/__init__.py	0	0	No	da39a3ee5e

Appendix C — Logs Schema + Samples

C.1 chat_logs.ndjson

Field	dtype
qid	object
question	object
answer	object
debug	object
contexts	object
img_path	bool
confidence	float64
feedback	object
timestamp	object
ts	datetime64[ns, UTC]
date	object

Sample record (excerpt):


```
{
 "qid": "49caea41-a99a-4fe5-8a27-28dbd79007fd",
 "question": "recall vs precision",
 "answer": "Recall measures the proportion of actual positives that were correctly identified, indicating that high recall means most positive instances are captured by the model [C6.pdf:24]. Precision measures the proportion of positive identifications that were actually correct, with high precision indicating a low rate of false\u2026",
 "debug": {
  "text_openai": {
```

```
    "637": 0.5554543733596802,
    "636": 0.49384650588035583,
    "638": 0.4410175085067749,
    "642": 0.37986648082733154,
    "632": 0.3786030113697052,
    "639": 0.37793487310409546,
    "633": 0.3634006381034851,
    "620": 0.3510317802429199,
    "634": 0.3496254086494446,
    "625": 0.34751883149147034
   },
   "img2img": {},
   "text2img": {
    "C6.pdf:48": 0.9999997117280004,
    "C6.pdf:25": 0.41028715926970327,
    "C3.pptx.pdf:114": 0.5544074770838178,
    "C5.pdf:73": 0.7458494597494455,
    "C6.pdf:8": 0.7280364122128162,
    "C5.pdf:72": 0.6796556360346927,
    "C6.pdf:26": 0.632976097062336,
    "C5.pdf:61": 0.6049657734654853,
    "C6.pdf:4": 0.12282116815885863,
    "C4.pptx.pdf:110": 0.10783102417852569,
    "C4.pptx.pdf:97": 0.5319004100957468,
    "C6.pdf:68": 0.5287468585558843,
    "C6.pdf:60": 0.09820821655915082,
    "C6.pdf:30": 0.5144590028210303,
    "C3.pptx.pdf:50": 0.5109962795615735,
    "C3.pptx.pdf:65": 0.07503258914955724,
    "C4.pptx.pdf:146": 0.5019618062662268,
    "C4.pptx.pdf:143": 0.06689034652023987,
    "C6.pdf:10": 0.4856974712203651,
    "C4.pptx.pdf:27": 0.05412076175299356,
    "C6.pdf:59": 0.05259292015499809,
    "C4.pptx.pdf:134": 0.4659574118478643,
    "C4.pptx.pdf:23": 0.024586718847742224,
    "C4.pptx.pdf:151": 0.44101882819738386,
    "C4.pptx.pdf:18": 0.01528504623594602,
    "C4.pptx.pdf:37": 0.009088351332076083,
    "C4.pptx.pdf:108": 0.0
   }
  },
  "contexts": [
   {
    "pdf": "C6.pdf",
    "page": 25,
```

```
    "score": 0.8064671245334837
   },
   {
    "pdf": "C6.pdf",
    "page": 24,
    "score": 0.7864855369735628
   }
  ],
  "img_path": false,
  "confidence": 0.8064671245334837,
  "feedback": "helpful",
  "timestamp": "2025-11-08T19:43:29.955186",
  "ts": "2025-11-08 19:43:29.955186+00:00",
  "date": "2025-11-08"
}
```


C.2 youtube_queries_log.json

Field	dtype
timestamp	object
question	object
lecture_filter	object
results	object
num_results	int64
has_results	bool
ts	datetime64[ns, UTC]
date	object

Sample record (excerpt):


```
{
  "timestamp": "2025-11-14T17:06:45.006729",
  "question": "svm",
  "lecture_filter": null,
  "results": [],
  "num_results": 0,
  "has_results": false,
  "ts": "2025-11-14 17:06:45.006729+00:00",
  "date": "2025-11-14"
}
```

Appendix D — Multimodal Index Artifacts (SmartTA_RAG/data/index)

Index footprint (MB)	34.1
PDFs / pages	6 PDFs · 698 pages
Chunks / images	734 chunks · 2076 images
Embedding shapes	text (734, 3072) · img (2076, 768)
File	Size (MB)
text.faiss	8.61
X_text.npy	8.60
images_openai_clip_vit_large_patch14_336.faiss	6.10
X_img_openai_clip_vit_large_patch14_336.npy	6.08
text_clip.faiss	2.16
X_text_clip.npy	2.15
image_meta.jsonl	0.23
chunks.jsonl	0.17
chunk_meta.jsonl	0.03
page_to_chunk_ids.pkl	0.01